

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2013

Huấn luyện viên: Hu Weidong, Tang Wenbin

Biên tập: Hu Weidong

China Computer Federation, 2013

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013

IOI China candidate team papers / National training team 2013 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2013. Bản dịch này chuyển ngữ thông tin đầu sách, mục lục và sáu bài đầu tiên: *Kế hoạch lên đỉnh*, của Peng Tianyi, *Lấy số trên lưới*, của Wang Kangning, *Thảo luận bài Two strings*, của Luo Gan, và *Catch The Penguins*, của Zhang Wentao, và *Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu*, của Luo Jianqiao, và *Kỹ thuật meet in the middle trong bài toán tìm kiếm*, của Qiao Mingda. Trạng thái hiện tại: sáu bài đầu đã được dịch; các bài còn lại mới có mục lục. Lớp văn bản của phần thân bị lỗi ảnh xạ phông chữ, nên phần bài viết được đối chiếu từ OCR trang render và các công thức, ví dụ còn đọc được trong lớp văn bản.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013*. Huấn luyện viên là Hu Weidong và Tang Wenbin; biên tập là Hu Weidong. Đơn vị xuất bản ghi trên bìa là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Kế hoạch lên đỉnh	Peng Tianyi, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
9	Lấy số trên lưới	Wang Kangning, Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm
17	Thảo luận bài <i>Two strings</i>	Luo Gan, Trường Học Quân Hàng Châu, Chiết Giang
23	<i>Catch The Penguins</i>	Zhang Wentao, Trường Học Quân Hàng Châu, Chiết Giang
27	Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu	Luo Jianqiao, Trường THPT số 80 Bắc Kinh
43	Kỹ thuật <i>meet in the middle</i> trong bài toán tìm kiếm	Qiao Mingda, Trường Ngoại ngữ Nam Kinh, Giang Tô
57	Phân tích cơ sở và ứng dụng của xác suất trong thi tin học	Hu Yuanming, Trường Trung học Dương Châu, Giang Tô
71	Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu	Xu Haoran, Trường Ngoại ngữ Nam Kinh, Giang Tô
85	Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học	Chen Lijie, Trường Ngoại ngữ Hàng Châu, Chiết Giang
99	Bàn về bài toán đếm trên vòng	Gao Shenghan, Trường Trung học cao cấp Thường Châu, Giang Tô
109	Ứng dụng của phương pháp chia khối	Wang Ziyu, Trường Trung học số 1 Shengli, Sơn Đông
121	Bàn về nguyên lý bao hàm–loại trừ	Wang Di, Trường Trung học số 7 Thành Đô, Tứ Xuyên

3 Trạng thái và ghi chú trích xuất

Các trang in trong mục lục lệch 4 trang so với trang vật lý của PDF: trang vật lý 5 là trang in 1. Bài *Kế hoạch lên đỉnh* dùng các trang in 1–8, tức trang vật lý 5–12. Bài *Lấy số trên lưới* dùng các trang in 9–16, tức trang vật lý 13–20. Bài *Thảo luận bài Two strings* bắt đầu ở trang in 17, tức trang vật lý 21; bài kế tiếp bắt đầu ở trang in 23, tức trang vật lý 27. Trang in 22 là trang trắng, nên phần dịch bài thứ ba lấy nội dung ở các trang in 17–21. Bài *Catch The Penguins* dùng các trang in 23–26, tức trang vật lý 27–30. Bài *Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu* bắt đầu ở trang in 27, tức trang vật lý 31; bài kế tiếp bắt đầu ở trang in 43, tức trang vật lý 47. Trang in 42 là trang trắng, nên phần dịch bài thứ năm lấy nội dung ở các trang in 27–41. Bài *Kỹ thuật meet in the middle trong bài toán tìm kiếm* bắt đầu ở trang in 43, tức trang vật lý 47; bài kế tiếp bắt đầu ở trang in 57, tức trang vật lý 61. Phần dịch bài thứ sáu lấy nội dung ở các trang in 43–56.

4 Kế hoạch lên đỉnh

Peng Tianyi

Trường THPT trực thuộc Đại học Sư phạm Hồ Nam

4.1 Bối cảnh bài toán

Leo núi là một môn vận động rất có ích cho sức khỏe thể chất và tinh thần. Nhưng khi leo núi, ta nên chọn chiến lược như thế nào để lên tới đỉnh núi nhanh nhất? Từ câu hỏi thú vị này, tác giả nghĩ ra bài toán dưới đây.

4.2 Mô tả bài toán

Giới hạn tài nguyên. Giới hạn thời gian: 2 giây. Giới hạn bộ nhớ: 512 MB.

Phát biểu. Trên mặt phẳng hai chiều, một dãy núi được xác định bởi một dãy đỉnh

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

Hai đỉnh kề nhau được nối bằng đoạn thẳng, với bảo đảm x_i tăng nghiêm ngặt. Như vậy ta thu được một dãy núi liên tục; giá trị y của mỗi điểm là độ cao của điểm đó.

Ta định nghĩa phần trong của dãy núi là phần kẹp giữa đường gấp khúc nối các đỉnh và trục x , không tính chính đường gấp khúc đó. Nếu đoạn thẳng nối đỉnh A và đỉnh B không đi qua phần trong của dãy núi, ta nói đỉnh A nhìn thấy đỉnh B , hoặc đỉnh B nhìn thấy đỉnh A .

Bây giờ pty xuất phát từ một đỉnh nào đó và muốn lên tới đỉnh cao nhất của dãy núi. Anh chỉ có thể đi trên đường gấp khúc giữa các đỉnh. Sau khi suy nghĩ, anh chọn cách leo như sau:

1. Đứng tại điểm xuất phát và nhìn sang hai bên. Nếu đỉnh núi cao nhất nhìn thấy được lúc này nằm bên trái, anh đi sang trái; ngược lại anh đi sang phải.
2. Trong lúc đi, pty vẫn quan sát đỉnh cao nhất nhìn thấy được ở hai bên. Một khi phát hiện một đỉnh cao hơn tất cả những đỉnh đã thấy trước đó, anh sẽ đi về phía đỉnh cao nhất lúc này.
3. Nếu có một thời điểm mà vị trí pty đang đứng cao hơn mọi đỉnh cao nhất nhìn thấy được ở hai bên, thì anh đã tới đỉnh cao nhất của dãy núi và quá trình leo núi kết thúc.

pty muốn biết: nếu dùng chiến lược trên và xuất phát từ từng đỉnh, quãng đường để tới đỉnh cao nhất lần lượt là bao nhiêu? Khoảng cách giữa hai điểm trên mặt phẳng là độ dài đoạn thẳng nối chúng.

Dữ liệu vào. Dòng đầu tiên chứa một số nguyên n , là số đỉnh của dãy núi.

Tiếp theo là n dòng; dòng thứ i chứa hai số nguyên x_i, y_i , biểu diễn tọa độ của đỉnh thứ i . Bảo đảm x_i tăng nghiêm ngặt, các y_i đôi một khác nhau trừ y_1, y_n , và x_i, y_i đều là số nguyên không âm. Bảo đảm $y_1 = y_n = 0$.

Dữ liệu ra. In ra n dòng, mỗi dòng một số thực.

Số thực ở dòng thứ i biểu diễn quãng đường từ đỉnh thứ i tới đỉnh cao nhất. Nếu sai số giữa kết quả in ra và đáp án chuẩn không vượt quá 10^{-2} , test đó được điểm tối đa; ngược lại được 0 điểm.

Ví dụ vào.

6
0 0
1 6
2 4
3 1
5 5
6 0

Ví dụ ra.

6.08
0.00
2.24
6.52
9.87
14.97

Giải thích ví dụ. Các lộ trình là:

$A : A \rightarrow B,$
 $B : B,$
 $C : C \rightarrow B,$
 $D : D \rightarrow G \rightarrow D \rightarrow C \rightarrow B,$
 $E : E \rightarrow D \rightarrow C \rightarrow B,$
 $F : F \rightarrow E \rightarrow D \rightarrow C \rightarrow B.$

Khi xuất phát từ D , đỉnh cao nhất nhìn thấy được ban đầu là E . Khi đi tới điểm G , ptý phát hiện đỉnh cao hơn là B , nên quay đầu và tiếp tục đi về phía B . Với các điểm xuất phát còn lại, lộ trình không cần đổi hướng.

Quy mô dữ liệu và quy ước. Mọi dữ liệu đều thỏa mãn $x_i, y_i \leq 100000$.

Test	Ràng buộc
1–4	$n \leq 20$
5–8	$n \leq 70$
9–10	$n \leq 100000$, và mọi đỉnh đều nhìn thấy trực tiếp đỉnh cao nhất
11–14	$n \leq 30000$
15–20	$n \leq 100000$

4.3 Phân tích bài toán

4.3.1 Đơn giản hóa bài toán

Bài toán yêu cầu tại mọi thời điểm khi leo núi ta đều phải quan sát đỉnh cao nhất hiện tại ở hai bên. Trước hết hãy tạm đơn giản hóa vấn đề: chỉ khi tới một đỉnh ta mới quan sát đỉnh cao nhất lúc đó. Nói cách khác, điểm quan sát tạm thời chuyển từ vô hạn nhiều điểm thành hữu hạn $O(n)$ đỉnh.

4.3.2 Tìm đỉnh cao nhất mà một đỉnh nhìn thấy

Gọi đỉnh cao nhất mà đỉnh i nhìn thấy được về bên trái là $L[i]$, và đỉnh cao nhất mà nó nhìn thấy được về bên phải là $R[i]$. Hãy tưởng tượng ta đứng tại một điểm nào đó; ban đầu tia nhìn nằm ngang về bên trái, sau đó ta từ từ ngẩng đầu lên cho tới khi tia nhìn không còn bị một ngọn núi nào che khuất. Khi đó ta tìm được $L[i]$.

Giả sử đỉnh hiện tại là A , còn $L[i]$ là B . Hai tính chất sau là hiển nhiên:

- i điểm đầu đều nằm phía dưới đoạn AB , theo nghĩa không nghiêm ngặt;
- A và B là hai điểm kề nhau cuối cùng trên bao lồi trên của i điểm đầu.

Vì A chắc chắn là điểm cuối cùng của bao lồi trên của i điểm đầu, B chính là điểm áp chót trên bao lồi trên đó.¹ Do đó ta chỉ cần dùng một stack để duy trì bao lồi của các điểm đầu tiên. Mỗi lần thêm điểm thứ $i + 1$, ta đồng thời duy trì tính chất của bao lồi. Vì tổng cộng chỉ có $O(n)$ đỉnh, ta có thể tìm toàn bộ $L[i]$ và $R[i]$ trong thời gian $O(n)$.

4.3.3 Xây dựng mô hình đồ thị

Cách làm vét cạn. Sau khi có $L[i]$ và $R[i]$, ta có một thuật toán khá thô: với từng đỉnh xuất phát, mô phỏng sự thay đổi của lộ trình. Độ phức tạp là $O(n^2)$ hoặc cao hơn. Điều này cho thấy ta cần tiếp tục khai thác phần thông tin bị tính lặp.

Dựng cây. Gọi $H[i]$ là độ cao của đỉnh thứ i , và đặt

$$T[i] = \max(H[L[i]], H[R[i]]).$$

Khi đó $T[i]$ biểu diễn độ cao của đỉnh cao nhất mà điểm i nhìn thấy được.

Khi ta xuất phát từ A , đến điểm đầu tiên B thỏa mãn

$$T[B] \geq T[A],$$

thì đoạn đường còn lại cần đi sẽ hoàn toàn giống như khi xuất phát từ B . Vì vậy khoảng cách từ A tới đỉnh cao nhất bằng khoảng cách từ B tới đỉnh cao nhất cộng thêm khoảng cách từ A tới B .

Lúc này cho B nối tới A bằng một cạnh có hướng, trọng số là độ dài AB . Làm như vậy với mọi đỉnh, ta thu được một cây: đỉnh cao nhất không có bậc vào, còn mọi đỉnh khác có bậc vào bằng 1. Tổng trọng số trên đường đi từ gốc cây tới một đỉnh bất kỳ chính là quãng đường từ đỉnh đó tới đỉnh cao nhất. Chỉ cần DFS một lần là có đáp án cho mọi đỉnh.

Vấn đề cần giải quyết. Với một điểm A , làm thế nào nhanh chóng tìm được điểm đầu tiên về bên trái hoặc bên phải thỏa mãn $T[B] \geq T[A]$? Đây là một bài toán cấu trúc dữ liệu rất kinh điển. Ta có thể giải bằng nhị phân đáp án kết hợp RMQ trên đoạn. Dưới đây là một cách có hằng số nhỏ hơn.

Lập danh sách liên kết hai chiều của toàn bộ đỉnh theo thứ tự từ trái sang phải. Sau đó duyệt tất cả các điểm theo thứ tự T tăng dần. Mỗi khi duyệt tới một điểm A , điểm đầu tiên bên trái thỏa mãn $T[B] \geq T[A]$ chính là hàng xóm bên trái của A còn tồn tại trong danh sách liên kết tại thời điểm đó. Nối cạnh tương ứng, rồi xóa A khỏi danh sách liên kết. Phía bên phải được xử lý tương tự. Độ phức tạp của thuật toán này chính là độ phức tạp sắp xếp.

¹Một kết luận đẹp như vậy thật sự làm tác giả ngạc nhiên.

4.3.4 Quay lại bài toán ban đầu

Tới đây ta đã giải xong phiên bản đơn giản hóa trong thời gian sắp xếp $O(n \log n)$. Bây giờ hãy quay lại bài toán ban đầu.

Ta có khả năng quan sát đỉnh cao nhất tại mọi thời điểm. Điều này làm bài toán thay đổi như thế nào?

Giả sử hiện tại ta xuất phát từ điểm A , đi sang phải, tới một điểm B trên một đoạn nào đó thì phát hiện một đỉnh cao hơn ở bên trái, nên quay sang đi về bên trái. Điểm rẽ B phải có những tính chất gì? Gọi đoạn chứa B là P_1P_2 . Khi đó B thỏa mãn:

- B là giao điểm giữa P_1P_2 và đường thẳng nối hai điểm nào đó bên trái A , ký hiệu là T_1, T_2 ;
- các điểm trước A đều nằm phía dưới đường thẳng T_1T_2 , theo nghĩa không nghiêm ngặt;
- T_1, T_2 là hai điểm kề nhau trên bao lồi trên của các điểm trước A .

Điều này có nghĩa: một điểm quan sát có ý nghĩa B sẽ là giao điểm đầu tiên giữa đường kéo dài của một cạnh nào đó trên bao lồi trên và đường gấp khúc của dãy núi. Vì tổng số cạnh được sinh ra trong quá trình duy trì bao lồi trên là $O(n)$, số điểm quan sát có ý nghĩa cũng là $O(n)$.²

Vậy làm thế nào nhanh chóng tìm ra các điểm quan sát đó?

Nếu đường thẳng T_1T_2 cắt đoạn P_1P_2 , điều này có nghĩa là khi P_2 được thêm vào bao lồi, T_2 sẽ bị bật ra; còn khi P_1 được thêm vào bao lồi thì T_2 chưa bị bật ra. Vì thế trong lúc duy trì bao lồi, ta chỉ cần lấy giao điểm giữa các đường thẳng liên quan tới điểm bị bật ra và điểm vừa chèn, là có thể thu được toàn bộ điểm quan sát. Độ phức tạp vẫn là $O(n)$.

Bài toán với hữu hạn điểm quan sát đã được giải ở trên. Vì thế toàn bộ bài toán được giải trọn vẹn trong thời gian $O(n \log n)$.

4.4 Tổng kết

4.4.1 Nguồn cảm hứng

Cảm hứng của bài toán đến từ suy nghĩ về việc leo núi trong đời sống thực. Dùng kiến thức thuật toán để phân tích một vấn đề thực tế không chỉ nâng cao năng lực phân tích thuật toán, mà còn phản ánh mục đích cốt lõi của việc học là “học để dùng”. Nó cũng có thể khơi dậy hứng thú đối với thi Tin học và làm ta thấy vui trong quá trình đó. Hy vọng cách suy nghĩ này đem lại gợi ý cho mọi người.

4.4.2 Tư duy giải bài

Nhìn lại quá trình giải bài: trước hết ta đơn giản hóa bài toán, biến vô hạn thành hữu hạn; tiếp đó chia bài toán thành các phần nhỏ hơn và lần lượt giải quyết. Đây đều là những phương pháp thường dùng trong quá trình làm bài.

Dưới đây là vài cảm nhận của tác giả về phương pháp giải bài, xin chia sẻ với mọi người.

Trong quá trình giải bài thông thường, ta cần dựng một chiếc cầu giữa điều kiện đã biết và vấn đề cần tìm. Có thể dùng công cụ đã học để suy luận xuôi từ điều kiện đã biết, xem ta còn thu được gì. Cũng có thể suy luận ngược từ vấn đề cần tìm, xem ta còn cần biết gì. Khi cây cầu suy luận xuôi và cây cầu suy luận ngược có một điểm trùng nhau, cây cầu nối từ điều kiện tới bài toán đã được dựng xong, và bài toán cũng được giải quyết.

Tuy nhiên phương pháp giải bài biến hóa muôn hình vạn trạng; không tồn tại bí kíp vạn năng nào. Điều đáng mừng là: khi số bài bạn đã giải càng nhiều, khả năng bạn giải được bài tiếp theo càng lớn.

²Đây cũng là một tính chất gây phấn khích.

Vì vậy, chỉ cần chăm luyện tập và chịu suy nghĩ, năng lực giải bài chắc chắn sẽ không ngừng bước lên những nấc thang mới.

5 Lấy số trên lưới

Wang Kangning

Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm

Tóm tắt

Đây là báo cáo lời giải cho một bài toán thú vị trong đợt làm đề đối kháng của đội tuyển quốc gia năm 2013.

5.1 Đề bài

5.1.1 Mô tả bài toán

Cho một bảng ô vuông $N \times N$. Mỗi ô chứa một số; số ở hàng i , cột j bằng $A_i \cdot B_j$.

Một người đi từ góc trên bên trái của một hình chữ nhật con tới góc dưới bên phải của hình chữ nhật đó. Mỗi bước chỉ được đi sang phải hoặc đi xuống một ô. Anh ta cộng các số đi qua trên đường đi và muốn tổng này nhỏ nhất.

Anh ta quyết định đi T lần. Lần thứ i , anh đi từ ô hàng P_i , cột Q_i tới ô hàng R_i , cột S_i . Hãy hỏi tổng nhỏ nhất của các số đi qua trong mỗi lần.

5.1.2 Dữ liệu vào

Dòng đầu tiên gồm hai số nguyên N, T , lần lượt là kích thước bảng và số lượng truy vấn.

Dòng thứ hai gồm N số nguyên $A_1, A_2, \dots, A_N$. Dòng này được sinh ngẫu nhiên.

Dòng thứ ba gồm N số nguyên $B_1, B_2, \dots, B_N$. Dòng này cũng được sinh ngẫu nhiên.

Tiếp theo là T dòng, mỗi dòng gồm bốn số nguyên P_i, Q_i, R_i, S_i , có ý nghĩa như trên và thỏa mãn $P_i \leq R_i, Q_i \leq S_i$.

5.1.3 Dữ liệu ra

In ra T dòng; mỗi dòng là một số nguyên, biểu diễn đáp án của truy vấn tương ứng.

5.1.4 Ví dụ vào

```
3 4
1 2 3
2 3 4
1 1 1 1
1 3 1 3
1 1 3 3
1 1 3 1
```

5.1.5 Ví dụ ra

2
4
29
12

5.1.6 Phạm vi dữ liệu

- Với 20% số test: $N \leq 100, T \leq 100$.
- Với 50% số test: $N \leq 3000, T \leq 1000$.
- Với 70% số test: $N \leq 50000, T \leq 20000$.
- Với 100% số test: $N \leq 200000, T \leq 50000, 1 \leq A_i, B_i \leq 10^6$.

Bảo đảm mọi truy vấn đều hợp lệ. Ngoài ví dụ, tất cả các giá trị A_i, B_i đều được sinh ngẫu nhiên.

Giới hạn tài nguyên. Giới hạn thời gian là 2 giây, giới hạn bộ nhớ là 512 MB.

5.2 Hướng giải

5.2.1 Thuật toán một

Với mỗi truy vấn, dùng quy hoạch động trực tiếp trên toàn bộ hình chữ nhật của truy vấn.

Độ phức tạp thời gian là $O(T \cdot N^2)$, độ phức tạp bộ nhớ là $O(N^2)$. Cách này chỉ kỳ vọng đạt khoảng 20 điểm.

5.2.2 Thuật toán hai

Thuật toán trên trả lời truy vấn quá chậm, nên ta cần tăng tốc phần truy vấn.

Xét tư tưởng chia để trị. Mỗi lần ta dùng đường giữa để chia hình chữ nhật thành hai nửa, dùng quy hoạch động tính đáp án từ từng điểm trên đường giữa tới từng điểm trong hình chữ nhật hiện tại, rồi đệ quy xử lý hai hình chữ nhật ở hai bên đường giữa. Nếu điểm đầu và điểm cuối của một truy vấn nằm ở hai phía của đường giữa, ta liệt kê điểm trên đường giữa mà đường đi đi qua; nếu không, truy vấn được đẩy xuống một hình chữ nhật con.

Nếu lần nào cũng cắt theo đường thẳng đứng, đặt độ phức tạp tiền xử lý là $\text{Time}(x)$, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(N^2x).$$

Khi đó độ phức tạp tiền xử lý là $O(N^3 \log N)$. Nếu luân phiên cắt theo chiều ngang và chiều dọc, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(x^3),$$

nên độ phức tạp tiền xử lý là $O(N^3)$.

Tổng độ phức tạp thời gian là $O(N^3 + TN)$, độ phức tạp bộ nhớ là $O(N^3)$. Chi phí tiền xử lý vẫn quá cao, vì vậy cần tìm phương án khác.

5.2.3 Thuật toán ba

Xét thuật toán chia khối. Nếu chia hình vuông thành các khối $S \times S$, ta tiền xử lý đáp án từ mỗi điểm tới mỗi điểm trên biên khối, đồng thời tiền xử lý đáp án giữa mọi cặp điểm trong cùng một khối.

Khi trả lời truy vấn, nếu hai điểm không nằm trong cùng một khối, ta liệt kê vị trí đầu tiên mà đường đi đi qua biên khối; nếu chúng nằm trong cùng một khối thì trả lời trực tiếp.

Độ phức tạp thời gian là

$$O\left(N^2\left(S + \frac{N}{S^2}\right) + T \cdot \frac{N}{S}\right),$$

và độ phức tạp bộ nhớ là

$$O\left(N^2\left(S + \frac{N}{S^2}\right)\right).$$

Độ phức tạp này cũng còn quá cao.

5.2.4 Cải tiến

Một nguyên nhân chính khiến các thuật toán trên kém hiệu quả là chúng chưa dùng tới điều kiện quan trọng trong đề: hai dãy A và B đều được sinh ngẫu nhiên.

Qua thử nghiệm và quan sát, ta thấy số điểm rẽ của đường đi tối ưu không nhiều. Hãy xét một điểm rẽ và hai đoạn đường đi kề với nó. Nếu điểm rẽ nằm ở một cột có trọng số không nhỏ hơn một cột nào đó bên trái trong vùng đang xét, đồng thời cũng không nhỏ hơn một cột nào đó bên phải, thì luôn tồn tại một cách biến đổi đường đi không tệ hơn và làm mất điểm rẽ đó.

Cụ thể, giả sử trong hình minh họa của bài gốc, cột thứ năm trong khung có trọng số không nhỏ hơn trọng số của cột ngoài cùng bên trái và cột ngoài cùng bên phải của khung. Ta xét hai cách biến đổi đường đi quanh điểm rẽ:

- Nếu trọng số của hàng phía trên trong vùng nhỏ hơn trọng số của hàng phía dưới, dùng biến đổi thứ nhất sẽ cho đáp án tốt hơn.
- Nếu trọng số của hàng phía trên lớn hơn trọng số của hàng phía dưới, dùng biến đổi thứ hai sẽ cho đáp án tốt hơn.
- Nếu hai trọng số hàng bằng nhau, dùng cách biến đổi nào thì đáp án cũng không xấu đi.

Như vậy ta có thể loại bỏ điểm rẽ đang xét.

Từ lập luận trên suy ra: với một điểm rẽ bất kỳ, trọng số của hàng hoặc cột chứa nó phải nhỏ hơn tất cả trọng số của các hàng hoặc cột ở một phía trong vùng truy vấn. Nói cách khác, nếu nhìn theo một phía của đoạn, điểm rẽ phải là một kỷ lục nhỏ mới.

Với dữ liệu ngẫu nhiên, xác suất để giá trị thứ i nhỏ hơn mọi giá trị đứng trước nó là $1/i$. Vì vậy số lượng điểm rẽ kỳ vọng là

$$1 + \frac{1}{2} + \dots + \frac{1}{N} = O(\log N).$$

5.2.5 Thuật toán bốn

Ta tìm vết cận mọi hàng và cột có thể chứa điểm rẽ. Sau đó dùng quy hoạch động để giải truy vấn, trong đó chỉ những hàng và cột có khả năng chứa điểm rẽ mới được xem là trạng thái hợp lệ.

Theo phân tích kỳ vọng ở trên, số hàng và số cột cần xét đều là $O(\log N)$; kỳ vọng của tích cũng bằng tích của các kỳ vọng, nên số trạng thái kỳ vọng là $O(\log^2 N)$. Ngoài ra, ta lưu tổng tiền tố của hai dãy A và B để tăng tốc các phép chuyển trạng thái dọc theo một đoạn thẳng.

Độ phức tạp thời gian kỳ vọng là

$$O\left(T \cdot (N + \log^2 N)\right) = O(TN).$$

Cách này kỳ vọng đạt khoảng 50 điểm.

5.2.6 Thuật toán năm

Nút thắt của thuật toán trước là việc tìm các hàng và cột có khả năng chứa điểm rẽ. Bài toán con này có thể phát biểu như sau: trong một khoảng của một dãy, tìm các số nhỏ hơn mọi số ở phía trước hoặc nhỏ hơn mọi số ở phía sau. Chỉ cần biết, với mỗi vị trí, phần tử đầu tiên ở phía sau hoặc phía trước nhỏ hơn nó là đủ. Việc này có thể thực hiện bằng hàng đợi đơn điệu.

Chẳng hạn, để tìm phần tử đầu tiên ở phía sau nhỏ hơn mỗi phần tử, ta quét dãy từ phải sang trái. Với phần tử hiện tại, loại khỏi đuôi hàng đợi đơn điệu mọi phần tử không nhỏ hơn nó. Khi đó phần tử đầu tiên ở phía sau nhỏ hơn nó chính là phần tử ở đuôi hàng đợi hiện tại; sau đó đưa phần tử hiện tại vào hàng đợi.

Tiền xử lý chỉ cần thời gian tuyến tính. Mỗi truy vấn sau đó được trả lời trong thời gian kỳ vọng $O(\log^2 N)$.

Độ phức tạp thời gian kỳ vọng là

$$O(N + T \log^2 N).$$

Cách này kỳ vọng đạt 100 điểm. Tới đây bài toán đã được giải quyết trọn vẹn.

5.3 Lời cảm ơn

Xin cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn cô Kong Weiling và thầy Gu Fangming vì sự chỉ dẫn.

Xin cảm ơn các bạn học đã giúp đỡ tôi.

6 Thảo luận bài *Two strings*

Luo Gan

Trường Học Quân Hàng Châu, Chiết Giang

6.1 Mô tả bài toán

Cho hai chuỗi A và B . Có năm loại thao tác:

1. Thêm một ký tự vào đầu chuỗi A .
2. Thêm một ký tự vào cuối chuỗi A .
3. Thêm một ký tự vào đầu chuỗi B .
4. Thêm một ký tự vào cuối chuỗi B .
5. Hỏi số lần chuỗi B hiện tại xuất hiện trong chuỗi A hiện tại.

Bảo đảm mọi ký tự đều là chữ cái thường, và ban đầu hai chuỗi A, B đều không rỗng.

6.1.1 Dữ liệu vào

Dòng thứ nhất và dòng thứ hai lần lượt là chuỗi A và chuỗi B ban đầu. Dòng thứ ba chứa một số nguyên m , biểu diễn số thao tác. Tiếp theo là m dòng, mỗi dòng biểu diễn một thao tác. Số đầu tiên của mỗi dòng là một số trong khoảng 1–5; nếu giá trị này khác 5, sau số đó sẽ có một chữ cái thường.

6.1.2 Dữ liệu ra

Với mỗi truy vấn, in ra một số nguyên trên một dòng, biểu diễn số lần chuỗi B xuất hiện trong chuỗi A .

6.1.3 Quy mô dữ liệu

- Với 10% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 200, số thao tác không vượt quá 10.
- Với 30% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 2000, số thao tác không vượt quá 1000.
- Với 100% dữ liệu, tổng độ dài cuối cùng của hai chuỗi A và B không vượt quá 200000, số thao tác không vượt quá 200000.

6.2 Ví dụ

6.2.1 Dữ liệu vào

ababc

a

7

5

4 b

5

3 a

1 a

5

5

6.2.2 Dữ liệu ra

2

2

1

1

6.2.3 Giải thích ví dụ

Lần hỏi	Chuỗi A	Chuỗi B	Số lần B xuất hiện trong A
Lần 1	ababc	a	2
Lần 2	ababc	ab	2
Lần 3	aababc	aab	1
Lần 4	aababc	aab	1

6.3 Thuật toán đơn giản

6.3.1 Mô phỏng

Mô phỏng từng thao tác. Với mỗi truy vấn, trực tiếp liệt kê các vị trí mà xâu B có thể xuất hiện trong xâu A , rồi so sánh trực tiếp.

6.3.2 Mô phỏng tối ưu bằng KMP

Trên cơ sở mô phỏng, với mỗi truy vấn ta dùng thuật toán KMP để thực hiện phép khớp xâu.

6.4 Tối ưu mô phỏng bằng xử lý ngoại tuyến

Đọc trước toàn bộ thao tác liên quan tới xâu A , rồi ghép chúng thành một “xâu A lớn”.

Khi đó, ở mỗi thời điểm truy vấn, xâu A hiện tại tương đương với một đoạn nào đó của xâu A lớn. Bài toán trở thành hỏi trong đoạn đó có bao nhiêu xâu B .

Mỗi khi gặp thao tác loại 3 hoặc loại 4, ta chạy lại KMP một lần và ghi nhận những vị trí khớp với xâu B . Như vậy truy vấn được chuyển thành: trong các vị trí đó, có bao nhiêu vị trí nằm trong đoạn từ L tới $R - \text{Len}(B) + 1$.

Vì L giảm dần còn R tăng dần, mỗi truy vấn có thể được giải trong thời gian $O(1)$.

6.4.1 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(n) \rightarrow O(1)$
Thao tác 2	$O(n) \rightarrow O(1)$
Thao tác 3	$O(n)$
Thao tác 4	$O(n)$
Thao tác 5	$O(n) \rightarrow O(1)$

6.5 Thuật toán xử lý ngoại tuyến bằng mảng hậu tố

6.5.1 Từ trực tuyến sang ngoại tuyến

Sử dụng tư tưởng ngoại tuyến ở trên: lấy kết quả cuối cùng của hai xâu A, B , ghép chúng lại với nhau, đồng thời chèn một ký tự phân cách ở giữa, chẳng hạn $+$. Sau đó dùng mảng hậu tố để duy trì quan hệ giữa các hậu tố.

Ví dụ:

Xâu A lớn: ABABC
Xâu B lớn: ABA
Xâu cuối cùng: ABABC+ABA

Sau khi sắp xếp mảng hậu tố:

A
 ABA
 ABC+ABA
 ABABC+ABA
 BA
 BABC+ABA
 BC+ABA
 C+ABA
 +ABA

6.5.2 Tìm khoảng hậu tố khả thi

Xâu B hiện tại xuất hiện trong chuỗi A ở các vị trí từ i tới $i + \text{Len}(B) - 1$ khi và chỉ khi độ dài tiền tố chung dài nhất Lcp của chuỗi B hiện tại và hậu tố thứ i của chuỗi A thỏa mãn

$$Lcp \geq \text{Len}(B).$$

Ta có thể dùng phương pháp nhân đôi để tiền xử lý mảng height, rồi tìm khoảng hậu tố này trong thời gian $O(\log n)$.

6.5.3 Thống kê vị trí bắt đầu hợp lệ trong chuỗi A

Ta định nghĩa một vị trí hợp lệ như sau: với vị trí i trong chuỗi A lớn, toàn bộ đoạn

$$[i, i + \text{Len}(B) - 1]$$

đều nằm trong chuỗi A hiện tại.

6.5.4 Duy trì bằng cây BIT

Ta có thể nhận thấy: mỗi thao tác bất kỳ chỉ làm thay đổi tính hợp lệ của nhiều nhất hai vị trí. Vì vậy có thể dùng cây BIT để duy trì, trên một đoạn của mảng hậu tố, có bao nhiêu vị trí hợp lệ.

6.5.5 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(\log n)$
Thao tác 2	$O(\log n)$
Thao tác 3	$O(\log n)$
Thao tác 4	$O(\log n)$
Thao tác 5	$O(\log n)$

6.6 Các điểm kiểm tra

- Suy nghĩ sâu về bản chất của bài toán.
- Nắm vững nhiều loại cấu trúc dữ liệu.
- Khả năng tối ưu nhiều loại thuật toán mô phỏng.
- Khả năng vận dụng thuật toán ngoại tuyến.

7 *Catch The Penguins*

Zhang Wentao
Trường Học Quân Hàng Châu, Chiết Giang

7.1 Tóm tắt bài toán

Cho tọa độ của N con chim cánh cụt trong không gian bốn chiều.

Có Q truy vấn. Mỗi truy vấn cho một điểm và hỏi có bao nhiêu chim cánh cụt có tọa độ ở mọi chiều đều nhỏ hơn hoặc bằng tọa độ tương ứng của điểm đó.

Ví dụ vào.

```
1
0 0 0 0
2
1 1 1 0
1 1 1 -1
```

Ví dụ ra.

```
1
0
```

Quy mô dữ liệu. Có tất cả 20 bộ dữ liệu.

Dữ liệu	Ràng buộc
1–2	$N, Q \leq 5000$
3–6	$N, Q \leq 10000$
7–14	$N \leq 30000, Q \leq 10000$
15–18	$N \leq 10000, Q \leq 30000$
19–20	$N, Q \leq 30000$

7.2 Thuật toán 0

Trực tiếp vét cạn. Với mỗi truy vấn, dùng một vòng lặp for để liệt kê từng chim cánh cụt, rồi kiểm tra xem cả bốn tọa độ của nó có đều nhỏ hơn hoặc bằng tọa độ của truy vấn hay không.

Độ phức tạp thời gian là $O(n^2)$.

Kỳ vọng đạt 10 điểm.

7.3 Tối ưu thuật toán 0

Sắp xếp riêng các chim cánh cụt theo từng chiều trong bốn chiều tọa độ.

Với mỗi truy vấn, trên mỗi chiều ta tìm nhị phân trong mảng đã sắp xếp tương ứng để biết số chim cánh cụt có tọa độ ở chiều đó nhỏ hơn hoặc bằng tọa độ của truy vấn. Lấy giá trị nhỏ nhất trong bốn kết quả này để chọn chiều đem ra liệt kê, như một tối ưu hằng số.

Kỳ vọng đạt 30 điểm.

7.4 Tiếp tục tối ưu?

Nếu muốn dùng thuật toán trực tuyến, dường như không có cấu trúc dữ liệu phù hợp để trả lời truy vấn tọa độ bốn chiều, nên rất khó tiếp tục theo hướng đó.

Ta hãy thử chuyển sang xét cách xử lý truy vấn ngoại tuyến.

7.5 Thuật toán chia khối

Khi đã có ý tưởng ngoại tuyến, kết hợp thêm tư tưởng chia khối thì không khó nghĩ ra một thuật toán có độ phức tạp tốt hơn một chút so với vét cạn.

7.5.1 Rời rạc hóa và xử lý ngoại tuyến

Sắp xếp riêng các truy vấn và các chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó liệt kê các truy vấn.

7.5.2 Với truy vấn đang được liệt kê

Thêm vào tất cả các chim cánh cụt có tọa độ chiều thứ nhất nhỏ hơn hoặc bằng tọa độ chiều thứ nhất của truy vấn.

Sắp xếp theo chiều thứ hai. Mỗi khi thêm $\sqrt{n}$ chim cánh cụt thì thực hiện một lần sắp xếp bằng vét cạn; phần còn lại cũng xử lý vét cạn.

Độ phức tạp của phần này là $O(n\sqrt{n})$.

7.5.3 Khi thực hiện một lần sắp xếp vét cạn

Giả sử số chim cánh cụt đã được thêm vào và đã sắp xếp là M .

Ta chia khối, sắp xếp theo tọa độ chiều thứ ba, rồi theo chiều thứ tư duy trì cây đoạn lưu trọng số tiền tố.

Cụ thể, có thể dùng cây đoạn hàm thức.

7.5.4 Truy vấn

Với các khối đầy đủ: dùng nhị phân cộng truy vấn.

Với các phần không trọn khối: xử lý vét cạn.

Tổng độ phức tạp là $O(n\sqrt{n} \log n)$.

Kỳ vọng đạt 60–95 điểm.

7.6 Thuật toán tốt hơn

Thông thường, xử lý chia khối là một phương án thay thế hiệu quả khi tồn tại thuật toán có độ phức tạp tốt hơn nhưng độ phức tạp tư duy lại khá cao. Ở đây, chỉ cần suy nghĩ thêm một chút là ta có thể tìm được một thuật toán chia để trị tốt hơn.

7.6.1 Bước 1

Sắp xếp chung các truy vấn và chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó thực hiện “trộn” theo tọa độ chiều thứ hai.

7.6.2 Bước 2

Giả sử dãy chỉ số sau khi sắp xếp là $id[L]$ tới $id[R]$, trong đó có phần tử là chim cánh cụt và có phần tử là truy vấn. Gọi MID là điểm cuối của đoạn bên trái.

7.6.3 Bước 3

Đệ quy xử lý đáp án của hai phần $L-MID$ và $MID+1-R$. Dĩ nhiên, khi L và R bằng nhau thì có thể trả về ngay.

7.6.4 Bước 4

Ta thấy trường hợp chưa được xử lý chỉ còn là ảnh hưởng của chim cánh cụt ở nửa trái, tức $L-MID$, lên truy vấn ở nửa phải, tức $MID+1-R$. Lúc này cả hai nửa đều đã được sắp xếp theo tọa độ chiều thứ hai (lý do được giải thích ngay sau đây), và thứ tự theo chiều thứ nhất của chúng không còn cần thiết nữa.

Ta trộn nửa trái và nửa phải theo tọa độ chiều thứ hai, tức gộp hai mảng tuyến tính đã có thứ tự. Khi tạo mảng mới, nếu phần tử được thêm là chim cánh cụt ở nửa trái, ta chèn điểm tạo bởi tọa độ chiều thứ ba và chiều thứ tư của nó vào một “tập hợp”. Nếu phần tử được thêm là truy vấn ở nửa phải, ta hỏi trong “tập hợp” có bao nhiêu điểm nằm ở góc dưới bên trái của điểm tạo bởi hai tọa độ chiều thứ ba và chiều thứ tư của truy vấn đó, rồi cộng số này vào đáp án của truy vấn.

7.6.5 Bước 5

Về cách cài đặt “tập hợp”, ta có thể dùng cây BIT (hoặc cây đoạn) lồng cây cân bằng.

Tổng độ phức tạp thời gian là $O(n \log^3 n)$.

7.7 Các điểm kiểm tra

- Chuyển đổi tư duy từ trực tuyến sang ngoại tuyến.
- Vận dụng các tư tưởng chia khối, chia để trị.
- Xử lý thứ tự dữ liệu một cách khéo léo.
- Nắm vững cấu trúc dữ liệu.

8 Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu

Luo Jianqiao

Trường THPT số 80 Bắc Kinh

Tóm tắt

Trong thi đấu và thực tiễn, làm thế nào để tổ chức và xử lý dữ liệu quy mô lớn một cách hiệu quả là một vấn đề quan trọng. Bài viết này chủ yếu thảo luận một lớp bài xử lý dữ liệu trong thi đấu, liên quan tới dãy tuyến tính và cấu trúc cây. Tác giả đưa vào khái niệm tư tưởng chia khối, khảo sát ứng dụng của nó

trong tổ chức dữ liệu, tiền xử lý dữ liệu và thiết kế thuật toán ngoại tuyến; đồng thời dùng ví dụ để minh họa cách chia dữ liệu thành khối nhằm thu được thuật toán có độ phức tạp thời gian thấp.

8.1 Dẫn nhập

Trong một lớp bài xử lý dữ liệu, ta cần thường xuyên thống kê thông tin về những phần tử thỏa mãn điều kiện nào đó trong một tập dữ liệu rất lớn, chẳng hạn số lượng hoặc giá trị cực trị. Các cấu trúc dữ liệu truyền thống, như cây đoạn, tổ chức toàn bộ dữ liệu trong tập theo một cách có thứ tự, để mỗi truy vấn có thể được tách thành tổng thông tin của một số phần có thứ tự, qua đó nâng cao hiệu quả.

Bài viết này giới thiệu một tư tưởng chia khối. Nó cũng làm dữ liệu có thứ tự và có tầng bậc, nhưng phương thức khác đi, khả năng mở rộng mạnh hơn và phạm vi áp dụng rộng hơn.

Cốt lõi của tư tưởng chia khối là chia một tập hợp thành một số tập con có quy mô nhỏ hơn. Nói chung, ta có xu hướng đưa các phần tử có tính chất gần nhau vào cùng một tập con, rồi tổ chức các phần tử trong mỗi tập con theo một cấu trúc nhất định.

Trong các bài xử lý dữ liệu, tư tưởng chia khối có phạm vi ứng dụng rộng vì cách chia khối có vài tính chất tốt:

- Quy mô của mỗi tập con nhỏ, nên có thể dùng thuật toán có độ phức tạp cao theo quy mô tập con.
- Số lượng tập con ít, nên có thể dùng thuật toán liên quan tới quy mô toàn cục để xử lý riêng từng tập con.
- Các phần tử trong mỗi tập con có tính chất chung, nên có thể thiết kế thuật toán có tính nhắm đích cho từng tập con.

Để thấy hai tính chất đầu mâu thuẫn với nhau: để cân bằng, ta không thể để tập con quá lớn, nhưng cũng không thể để số tập con quá lớn. Đây là lý do độ phức tạp thời gian của các thuật toán chia khối thường liên quan tới $\sqrt{N}$.

Bài viết chủ yếu bàn về ứng dụng của chia khối trong các bài xử lý dữ liệu liên quan tới dãy tuyến tính và cấu trúc cây. Mục 2 giới thiệu phương pháp chia khối chung trong các cấu trúc dữ liệu điển hình. Mục 3 giới thiệu ứng dụng của chia khối trong tiền xử lý dữ liệu. Mục 4 giới thiệu ứng dụng của chia khối khi thiết kế thuật toán ngoại tuyến.

8.2 Chia khối dữ liệu

Mục này giới thiệu phương pháp chia khối chung cho hai cấu trúc dữ liệu thường gặp là dãy tuyến tính và cây, rồi lần lượt đưa ra ví dụ dùng chia khối để giải.

8.2.1 Chia khối dãy tuyến tính

Trong dãy tuyến tính, mọi phần tử của tập dữ liệu được sắp xếp theo một thứ tự nhất định. Một bài điển hình là thường xuyên thống kê “tổng” thông tin của các phần tử thỏa mãn một tính chất nào đó trong một dãy con liên tiếp; ở đây “tổng” nghĩa là đáp án có thể được tách trực tiếp hoặc gián tiếp thành tổng đáp án của các phần khác nhau. Trước hết ta giới thiệu một cách chia khối thường dùng, sau đó minh họa cách sử dụng nó để giải hiệu quả loại bài này.

Cách chia khối như sau: bắt đầu từ phần tử đầu tiên của dãy, cứ mỗi S phần tử liên tiếp tạo thành một khối. Nếu cuối cùng còn ít hơn S phần tử, cho các phần tử còn lại tạo thành một khối.

Tính chất 2.1. Giả sử N là độ dài dãy. Khi dùng cách trên, số khối không vượt quá $\lceil N/S \rceil + 1$.

Tính chất 2.2. Giả sử một truy vấn hỏi thông tin của dãy con liên tiếp từ phần tử thứ L tới phần tử thứ R , gọi tắt là đoạn $[L, R]$. Khi đó, ngoài các phần tử nằm trong những khối được chứa trọn vẹn, số phần tử của đoạn $[L, R]$ còn dính tới không vượt quá $2S$.

Theo hai tính chất trên, nếu ta có thể duy trì ở tầng khối tổng thông tin của toàn bộ phần tử trong một khối, thì chỉ cần chọn kích thước khối S thích hợp, với mọi đoạn ta đều có thể nhanh chóng tìm ra những khối trọn vẹn và những phần tử thừa, rồi gộp chúng để thu được thông tin của cả đoạn.

Ví dụ một. Cho một dãy gồm N số nguyên. Cần thực hiện M thao tác thuộc hai loại:

- ADD D_i X_i : bắt đầu từ số thứ nhất, cứ cách D_i vị trí thì cộng thêm X_i vào phần tử đó.
- QUERY L_i R_i : hỏi tổng các phần tử từ vị trí L_i tới R_i .

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$; ở mọi thời điểm, giá trị tuyệt đối của mỗi số trong dãy không vượt quá 10^9 .

Phân tích thuật toán. Trong trường hợp xấu nhất, mỗi thao tác ADD và QUERY đều có thể liên quan tới N phần tử, nên mô phỏng đơn thuần sẽ rất chậm. Ta xét chia dãy thành khối và ghi ở tầng khối tổng các phần tử trong từng khối.

Bổ đề 2.1. Nếu đặt kích thước khối là $\sqrt{N}$, mỗi truy vấn có thể xử lý trong $O(\sqrt{N})$ thời gian.

Cách làm cụ thể là khi truy vấn, liệt kê từng khối; nếu khối nằm trọn trong đoạn truy vấn thì cộng trực tiếp tổng của khối vào đáp án. Số phần tử còn lại không vượt quá $2\sqrt{N}$, nên có thể liệt kê trực tiếp.

Tuy nhiên, độ phức tạp của thao tác sửa vẫn chưa được cải thiện. Lúc này ta tiếp tục dùng tư tưởng chia khối và chú ý tới sự thật sau:

Bổ đề 2.2. Khi và chỉ khi thao tác ADD có $D_i < \sqrt{N}$, số phần tử cần cập nhật mới vượt quá $\sqrt{N}$.

Vì vậy ta phân loại thao tác ADD. Nếu $D_i \geq \sqrt{N}$, cập nhật trực tiếp các phần tử bị ảnh hưởng và tổng khối tương ứng, độ phức tạp là $O(\sqrt{N})$. Nếu $D_i < \sqrt{N}$, chỉ cần dùng mảng để ghi tổng lượng sửa của mọi thao tác ADD có cùng D_i ; lúc này một thao tác sửa là $O(1)$.

Khi đó, giá trị của từng phần tử và tổng từng khối chỉ xét loại sửa thứ nhất. Khi truy vấn, ta còn phải liệt kê mọi D_i thuộc loại thứ hai và tính ảnh hưởng của tổng lượng sửa với D_i đó lên đáp án. Vì số D_i thuộc loại thứ hai không vượt quá $\sqrt{N}$, độ phức tạp một truy vấn vẫn là $O(\sqrt{N})$.

Định lý 2.3. Bài toán trên có thể giải trong thời gian $O((N + M)\sqrt{N})$.

8.2.2 Chia khối cấu trúc cây

Trong cấu trúc cây, các đỉnh kề nhau có tương quan rất mạnh. Một ý tưởng trực quan là mong muốn chia cây thành một số khối liên thông có quy mô nhỏ. Dưới đây là một cách chia khối trên cây: với S thích hợp, nó có thể chia cây thành $O(N/S)$ khối liên thông kích thước $O(S)$, trong đó N là số đỉnh của cây.

Trước hết cần khái niệm dãy DFS. Với cây vô căn, có thể chọn tùy ý một đỉnh làm gốc để biến nó thành cây có gốc.

Định nghĩa 2.1 (Dãy DFS của cây). Duy trì một dãy, ban đầu rỗng. Từ gốc, thực hiện duyệt sâu; mỗi khi gặp một đỉnh lúc vào stack hoặc ra khỏi stack thì thêm nó vào cuối dãy. Dãy thu được sau khi duyệt xong gọi là dãy DFS.

Như vậy, với một cây bất kỳ ta thu được một dãy DFS có $2N$ phần tử; mỗi đỉnh của cây tương ứng với hai phần tử trong dãy. Dãy DFS có những tính chất tốt.

Bổ đề 2.4. Trong dãy DFS, quan hệ giữa hai phần tử kề nhau chỉ có ba loại: hoặc là cùng một đỉnh, hoặc là quan hệ cha con, hoặc là quan hệ anh em. Loại thứ ba xảy ra khi và chỉ khi trong DFS, phần tử trước là đỉnh được đưa ra khỏi stack, còn phần tử sau là đỉnh vừa được đưa vào stack.

Từ bổ đề 2.4 có thể chứng minh tính chất sau.

Bổ đề 2.5. Một dãy con liên tiếp từ hạng u tới hạng v trong dãy DFS vừa đúng tương ứng với một đường đi từ đỉnh ứng với hạng u tới đỉnh ứng với hạng v . Ngoài LCA của hai đầu mút ra, mọi đỉnh trên đường đi đều xuất hiện ít nhất một lần trong dãy con đó.

Từ đó có kết luận ta cần:

Định lý 2.6. Dùng phương pháp chia khối của mục trước để chia dãy DFS của cây thành các khối kích thước S . Với mỗi khối của dãy, tập các đỉnh tương ứng với toàn bộ phần tử trong khối cùng LCA của chúng tạo thành một khối liên thông trên cây. Số khối liên thông không vượt quá $2N/S$.

Phương pháp chia khối này cài đặt đơn giản, đồng thời có tính chất chung của tư tưởng chia khối: mỗi khối không lớn và số khối cũng không nhiều. Đáng tiếc là có thể tồn tại một số đỉnh xuất hiện trong nhiều hơn hai khối liên thông.

Ví dụ hai. Cho một cây N đỉnh và một số nguyên K . Mỗi cạnh có trọng số. Cần tính với mỗi đỉnh i , trong các khoảng cách từ $N - 1$ đỉnh còn lại tới đỉnh i , giá trị nhỏ thứ K là bao nhiêu.

Quy mô dữ liệu: $1 \leq K < N \leq 50000$; trọng số cạnh là số nguyên có giá trị tuyệt đối nhỏ hơn 1000.

Phân tích thuật toán. Thuật toán đơn giản là lần lượt chọn mỗi đỉnh làm gốc, tính khoảng cách từ mọi đỉnh khác tới nó, rồi sắp xếp để lấy đáp án. Độ phức tạp thời gian là $O(N^2 \log N)$. Nếu dùng sắp xếp cơ sở, có thể giảm xuống $O(N^2)$.

Ta chia cây theo phương pháp ở trên thành một số khối liên thông kích thước $O(S)$. Xét việc lợi dụng tính liên quan giữa các đỉnh kề nhau để nhanh chóng tính đáp án khi từng đỉnh trong cùng một khối liên thông được chọn làm gốc.

Bổ đề 2.7. Giả sử u là một đỉnh bất kỳ không nằm trong khối liên thông hiện tại. Dù chọn đỉnh nào trong khối làm gốc, trên đường từ u tới gốc, đỉnh đầu tiên nằm trong khối là không đổi.

Theo bổ đề 2.7, ta phân loại mọi đỉnh ngoài khối theo đỉnh đầu tiên trong khối mà đường đi tới gốc đi qua, gọi tắt là tổ tiên gần nhất trong khối. Rõ ràng các đỉnh ngoài khối được chia thành nhiều nhất S loại.

Bổ đề 2.8. Giả sử u và v là hai đỉnh bất kỳ không nằm trong khối hiện tại và thuộc cùng một loại. Gọi p là tổ tiên gần nhất trong khối của chúng. Khi chọn bất kỳ đỉnh r nào trong khối làm gốc, quan hệ thứ tự giữa khoảng cách từ u tới r và từ v tới r đúng bằng quan hệ thứ tự giữa khoảng cách từ u tới p và từ v tới p .

Vì vậy, với mỗi loại, ta sắp xếp các đỉnh của loại đó theo khoảng cách tới tổ tiên gần nhất trong khối từ nhỏ tới lớn. Sau đó liệt kê từng đỉnh trong khối làm gốc và lần lượt tính đáp án.

Định lý 2.9. Khi chọn một đỉnh r bất kỳ trong khối liên thông làm gốc, có thể trong $O(S \log^2 N)$ thời gian tính được giá trị nhỏ thứ K trong các khoảng cách từ các đỉnh khác tới r .

Cách làm cụ thể như sau. Để tính giá trị nhỏ thứ K , ta nhị phân đáp án. Giả sử giá trị nhị phân là X . Từ r , liệt kê mọi đỉnh trong khối, tính khoảng cách của chúng tới r và đếm có bao nhiêu khoảng cách không vượt quá X . Với mỗi đỉnh p trong khối, giả sử khoảng cách từ p tới r là D_p ; trong bảng có thứ tự của loại nhận p làm tổ tiên gần nhất trong khối, nhị phân vị trí lớn nhất có giá trị không vượt quá $X - D_p$, qua đó tính số đỉnh ngoài khối thuộc loại này và cách r không quá X .

Sau khi xử lý mọi p , giả sử số đỉnh cách r không quá X là cnt . Nếu $\text{cnt} < K$, đáp án thật lớn hơn X ; ngược lại đáp án thật không lớn hơn X . Mỗi lần nhị phân thu hẹp một nửa phạm vi đáp án, do đó đạt độ phức tạp $O(S \log^2 N)$.

Bổ đề 2.10. Tính cả thời gian sắp xếp và số đỉnh trong khối, ta có thể tính đáp án cho mọi đỉnh trong một khối liên thông trong

$$O(N \log N + S^2 \log^2 N)$$

thời gian.

Định lý 2.11. Nếu chọn kích thước khối $S = \Theta(\sqrt{N / \log N})$, độ phức tạp thời gian tổng cộng của thuật toán là $O(N^{1.5} \log^{1.5} N)$.

8.2.3 Mở rộng thêm

Bạn đọc quan tâm có thể tự suy nghĩ cách mở rộng phương pháp chia khối dãy tuyến tính lên trường hợp d chiều, cũng như việc phương pháp chia khối cây có áp dụng được cho các cấu trúc phức tạp hơn hay không.

8.3 Chia khối và tiền xử lý

Tiền xử lý nghĩa là khi duy trì một tập dữ liệu, trước mọi thao tác truy vấn ta xử lý dữ liệu trước, thường ghi lại nhiều thông tin phụ, rồi khi truy vấn thì dùng thông tin đã xử lý để tăng hiệu quả thời gian. Nếu có thao tác sửa, còn phải xét ảnh hưởng của sửa lên thông tin đã tiền xử lý.

Kết hợp tư tưởng chia khối với tiền xử lý có thể giải hiệu quả một lớp bài truy vấn tập hợp rất rộng. Mục này trước hết nêu các tính chất mà loại bài có thể tối ưu bằng tiền xử lý cần thỏa mãn, sau đó giới thiệu chi tiết phương pháp tiền xử lý chung trên dãy và trên cây.

8.3.1 Điều kiện để tiền xử lý

Giả sử tập dữ liệu đang duy trì là tập S .

Điều kiện 3.1 (Có thể tăng lượng). Giả sử hai lần truy vấn tương ứng với hai tập con A và B của tập S . Độ phức tạp để chuyển thông tin liên quan của tập truy vấn A thành thông tin liên quan của tập truy vấn B là một hàm theo kích thước hiệu đối xứng của A và B , tức

$$(A - B) \cup (B - A).$$

Nói cách khác, độ phức tạp chuyển trạng thái chỉ phụ thuộc vào số phần tử thuộc hiệu đối xứng.

Gọi $\mathcal{P}$ là tập hợp mọi tập con của S có thể xuất hiện trong truy vấn.

Điều kiện 3.2 (Có thể tiền xử lý). Với các số nguyên K và L cho trước, tồn tại một tập con $\mathcal{Q} \subseteq \mathcal{P}$ có kích thước không vượt quá K , sao cho với mọi phần tử $X \in \mathcal{P}$, tồn tại một phần tử $Y \in \mathcal{Q}$ mà kích thước hiệu đối xứng giữa X và Y không vượt quá L .

Nếu truy vấn trên tập dữ liệu thỏa mãn điều kiện 3.1, đồng thời tồn tại K, L thích hợp để tập dữ liệu thỏa mãn điều kiện 3.2, ta có thể dùng phương pháp chia khối và tiền xử lý để tăng hiệu quả thời gian của thao tác truy vấn.

Khi đó ta tìm tập $\mathcal{Q}$ trong điều kiện 3.2 và tiền xử lý thông tin liên quan của các phần tử trong $\mathcal{Q}$. Mỗi khi truy vấn phần tử X , tìm phần tử $Y \in \mathcal{Q}$ tương ứng. Vì thông tin của Y đã được tiền xử lý, và truy vấn thỏa mãn điều kiện 3.1, ta có thể nhanh chóng chuyển thông tin của Y thành thông tin của X trong $O(f(L))$ thời gian.

8.3.2 Tiền xử lý trên dãy tuyến tính

Mục nhỏ này xét một lớp bài trên dãy tuyến tính: thường xuyên truy vấn thông tin của một dãy con liên tiếp và thỏa mãn điều kiện tiền xử lý. Trước hết là phương pháp tiền xử lý, sau đó là một ví dụ.

Ta chia dãy thành khối. Giả sử kích thước mỗi khối là S , số khối là $O(N/S)$. Theo điều kiện 3.1, với bất kỳ đoạn có ℓ phần tử, ta có thể thu được thông tin liên quan trong thời gian $O(f(\ell))$. Do đó ta tiền xử lý thông tin liên quan giữa hai khối bất kỳ như sau.

Lấy một khối A bất kỳ, xét việc lần lượt tính thông tin giữa khối A và từng khối B sau nó theo thứ tự vị trí trong dãy. Sau khi tính được thông tin từ khối A tới khối B_i , có thể trong $O(f(S))$ thời gian chuyển thành thông tin từ khối A tới khối B_{i+1} .

Định lý 3.1. Chỉ xét các khối B_i nằm sau khối A , nếu không tính độ phức tạp ghi thông tin, có thể trong $O(Nf(S)/S)$ thời gian tính thông tin giữa khối A và mọi khối B_i .

Nếu độ phức tạp ghi thông tin giữa hai khối bất kỳ có thể bỏ qua, ta liệt kê khối A và hoàn thành toàn bộ quá trình chia khối, tiền xử lý trong $O(N^2f(S)/S^2)$ thời gian.

Theo tính chất 2.2, thông tin của một đoạn bất kỳ $[L, R]$ có thể được suy ra từ thông tin của một dãy con giữa hai khối đã tiền xử lý bằng cách thêm không quá $2S$ phần tử. Vì vậy:

Định lý 3.2. Với bài thống kê đoạn trên dãy tuyến tính, nếu bài toán thỏa mãn điều kiện tiền xử lý, trước hết chia khối và tiền xử lý dãy; sau đó mỗi truy vấn đoạn $[L, R]$ có thể trả lời trong $O(f(S))$ thời gian.

Ví dụ ba. Cho một dãy gồm N số, mỗi số là một số nguyên trong khoảng 1 tới N . Cần trả lời M truy vấn. Mỗi truy vấn cho ba số nguyên L_i, R_i, K_i , hỏi trong đoạn $[L_i, R_i]$ có bao nhiêu số xuất hiện đúng K_i lần.

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$.

Phân tích thuật toán. Trong bài này, đáp án của các đoạn kề nhau không thể nhanh chóng gộp thành đáp án của đoạn hợp bằng cách ghi vài thông tin phụ, nên không thể dùng những cấu trúc dữ liệu như cây đoạn để duy trì. Nhưng truy vấn có tính chất sau:

Bổ đề 3.3. Với hai đoạn bất kỳ $[L_1, R_1]$ và $[L_2, R_2]$, nếu ta đã ghi số lần xuất hiện trong $[L_1, R_1]$ của từng số từ 1 tới N , cũng như số lượng các số có tần suất 1 tới N , thì có thể trong

$$O(|L_1 - L_2| + |R_1 - R_2|)$$

thời gian chuyển chúng thành thông tin tương ứng của đoạn $[L_2, R_2]$.

Vì vậy có thể dùng phương pháp tiền xử lý. Chia dãy thành khối với kích thước mỗi khối là $O(\sqrt{N})$, số khối cũng là $O(\sqrt{N})$. Ta có thể trong $O(N\sqrt{N})$ thời gian tính số lần xuất hiện của mỗi số từ 1 tới N , và số lượng các số có tần suất 1 tới N , giữa hai khối bất kỳ. Nhưng nếu ghi toàn bộ thông tin như vậy thì dung lượng là $O(N^2)$, nên cần giảm lượng thông tin phải ghi.

Bổ đề 3.4. Đánh số các khối từ trái sang phải là 1 tới $\sqrt{N}$. Số lần xuất hiện của một số x trong các khối từ i tới j bằng số lần x xuất hiện trong j khối đầu trừ số lần x xuất hiện trong $i - 1$ khối đầu.

Ta chỉ duy trì số lần xuất hiện của mỗi số từ 1 tới N trong tiền tố từ 1 tới $\sqrt{N}$ khối, giảm độ phức tạp không gian xuống $O(N\sqrt{N})$. Việc này tương đương với duy trì số lần xuất hiện giữa hai khối bất kỳ.

Bổ đề 3.5. Số giá trị xuất hiện trong dãy ít nhất $\sqrt{N}$ lần không vượt quá $\sqrt{N}$.

Khi tiền xử lý, ta chỉ ghi số lượng các giá trị có số lần xuất hiện từ 1 tới $\sqrt{N}$ giữa hai khối. Do đó độ phức tạp ghi thông tin là $O(N\sqrt{N})$. Theo định lý 3.2, ta có thể trả lời đúng mọi truy vấn có $K_i < \sqrt{N}$ trong $O(\sqrt{N})$ thời gian.

Với truy vấn có $K_i \geq \sqrt{N}$, ta xử lý trước tổng tiền tố tần suất của mọi giá trị xuất hiện nhiều hơn $\sqrt{N}$ lần. Khi truy vấn, liệt kê những giá trị này trong $O(\sqrt{N})$ thời gian, và $O(1)$ để kiểm tra tần suất của từng giá trị trong đoạn truy vấn có bằng K_i hay không.

Định lý 3.6. Thuật toán trên có thể hoàn thành tiền xử lý trong $O(N\sqrt{N})$ thời gian. Mỗi truy vấn được trả lời trong $O(\sqrt{N})$ thời gian. Tổng độ phức tạp thời gian và không gian là $O((N + M)\sqrt{N})$.

8.3.3 Tiền xử lý trên cấu trúc cây

Mục nhỏ này xét một lớp bài trên cây: thường xuyên truy vấn thông tin của đường đi giữa hai đỉnh và thỏa mãn điều kiện tiền xử lý. Ta mở rộng phương pháp chia khối tiền xử lý từ dãy tuyến tính lên cây, mô tả cách đánh dấu các đỉnh then chốt trên cây, rồi đưa ra một ví dụ.

Bổ đề 3.7. Trong cây, đường đi đơn giữa hai đỉnh bất kỳ là duy nhất.

Bổ đề 3.8. Với hai đỉnh bất kỳ u, v trong cây có gốc, gọi p là LCA của chúng. Đường đi từ u tới v vừa đúng bằng đường đi từ u tới p và đường đi từ p tới v .

Ta muốn đánh dấu một số đỉnh của cây làm đỉnh then chốt, đồng thời tiền xử lý đường đi giữa mọi cặp đỉnh then chốt, sao cho mọi đường đi đủ dài trên cây đều chứa một đường đi đã được xử lý, còn phần còn lại ngoài đường đi đó có số đỉnh ít. Dưới đây là một thuật toán tham lam.

Với cây có gốc, liệt kê các đỉnh theo thứ tự độ sâu giảm dần và quyết định có đánh dấu chúng làm đỉnh then chốt hay không. Đánh dấu một đỉnh u khi và chỉ khi trong cây con của u tồn tại một đỉnh mà trên đường từ đỉnh đó tới u chưa có đỉnh then chốt, đồng thời số đỉnh trên đường này lớn hơn S . Ở đây S là số cho trước.

Bổ đề 3.9. Giả sử N là số đỉnh của cây. Thuật toán tham lam đánh dấu không quá N/S đỉnh then chốt.

Lý do là mỗi đỉnh then chốt đều là đỉnh then chốt đầu tiên trên đường tới gốc của ít nhất S đỉnh không then chốt.

Bổ đề 3.10. Giả sử độ sâu của gốc là 0. Với bất kỳ đỉnh nào có độ sâu không nhỏ hơn S , trong $S + 1$ tổ tiên gần nhất của nó, kể cả chính nó, có ít nhất một đỉnh then chốt.

Hệ quả 3.11. Với bất kỳ đường đi đơn nào trong cây có số đỉnh lớn hơn $3S$, gọi u và v là hai đầu mút. Khoảng cách từ u tới đỉnh then chốt gần nhất trên đường đi không vượt quá $2S - 2$; tổng khoảng cách từ u và v tới các đỉnh then chốt gần nhất trên đường đi không vượt quá $3S - 3$.

Trong thuật toán tham lam đánh dấu đỉnh then chốt, chỉ cần ghi với mỗi đỉnh giá trị lớn nhất của số đỉnh trên một đường trong cây con của nó, chỉ gồm các đỉnh không then chốt và kết thúc tại nó, là có thể quyết định có đánh dấu nó hay không.

Định lý 3.12. Trong $O(N)$ thời gian, có thể đánh dấu $O(N/S)$ đỉnh then chốt trong cây. Nếu việc tiền xử lý giữa các cặp đỉnh then chốt có độ phức tạp $O((N/S)^2 f(N))$, thì với mọi đường đi đơn, sau khi bỏ phần đã được xử lý, số đỉnh còn lại là $O(S)$.

Ví dụ bốn. Cho một cây N đỉnh. Màu của mỗi đỉnh là một số nguyên từ 1 tới M .

Định nghĩa lợi ích du lịch từ đỉnh u tới đỉnh v là

$$\sum_i V_i W_{C_i},$$

trong đó i chạy qua các màu, C_i là số lần màu i xuất hiện trên đường đi đơn từ u tới v , và $W_0, W_1, \dots, W_N$ là dãy số nguyên.

Cần thực hiện Q thao tác thuộc hai loại:

- CHANGE $x_i \ y_i$: đổi màu của đỉnh x_i thành y_i .
- QUERY $u_i \ v_i$: hỏi lợi ích du lịch từ u_i tới v_i .

Quy mô dữ liệu: $1 \leq N, M, Q \leq 10^5$, $1 \leq V_i, W_i \leq 10^6$. Giới hạn thời gian của bài là 10 giây. Bài gốc là *Candy Park* của NOI Winter Camp 2013.

Phân tích thuật toán. Chọn một đỉnh bất kỳ làm gốc. Đặt $S = \Theta(N^{1/3})$, dùng phương pháp đánh dấu đỉnh then chốt để tiền xử lý trên cây, ghi số lần xuất hiện của các màu 1 tới M và lợi ích du lịch trên đường đi giữa mọi cặp đỉnh then chốt. Độ phức tạp tiền xử lý là $O(N^{5/3})$.

Với thao tác truy vấn đường đi từ u_i tới v_i , trước hết dùng nhân đôi hoặc thuật toán hiệu quả khác để tìm LCA của hai đỉnh. Sau đó lần lượt đi từ u_i và v_i dọc theo đường đi để tìm các đỉnh then chốt gần u_i và v_i nhất trên đường. Khi đó có thể dùng thông tin đã ghi và một số thao tác tăng lượng để thu được đáp án trong $O(N^{1/3})$ thời gian.

Với thao tác đổi màu đỉnh x_i , liệt kê mọi cặp đỉnh then chốt và kiểm tra x_i có nằm trên đường đi giữa chúng hay không. Nếu có thì cập nhật thông tin đã ghi.

Bổ đề 3.13. Đỉnh x_i nằm trên đường đi từ u tới v khi và chỉ khi LCA của u và v là tổ tiên của x_i , đồng thời x_i là tổ tiên của u hoặc v .

Nếu tiền xử lý thời điểm vào stack và ra khỏi stack trong DFS cho mỗi đỉnh, thì x là tổ tiên của y khi và chỉ khi thời điểm vào stack của y nằm giữa thời điểm vào và ra của x . Trong lúc tiền xử lý, ta ghi LCA của mọi cặp đỉnh then chốt, nên có thể $O(1)$ kiểm tra một đỉnh có nằm trên đường đi giữa chúng hay không. Vì vậy thao tác sửa có độ phức tạp $O(N^{2/3})$.

Định lý 3.14. Bài toán này có thể giải trong $O((N + Q)N^{2/3})$ thời gian.

8.4 Chia khối và thuật toán ngoại tuyến

Một số bài xử lý dữ liệu không yêu cầu trả lời từng truy vấn ngay lập tức, mà cho trước rất nhiều truy vấn và yêu cầu chương trình nhanh chóng tính toàn bộ kết quả rồi trả về một lần. Thuật toán ngoại tuyến là thuật toán biết trước nội dung của mọi truy vấn, vì vậy có thể giải chúng cùng nhau.

Trong bài ngoại tuyến, vì đã biết toàn bộ truy vấn, ta có thể tổ chức các truy vấn theo một cách nhất định, tận dụng đầy đủ thông tin trùng lặp giữa chúng. Về bản chất, ta xem toàn bộ truy vấn như một phần của tập dữ liệu, rồi dùng phương pháp xử lý tập dữ liệu để xử lý truy vấn.

Mục này dùng hai ví dụ để minh họa ứng dụng của tư tưởng chia khối khi thiết kế thuật toán ngoại tuyến.

Ví dụ năm. Cho một dãy gồm N số, mỗi số là một số nguyên trong khoảng 1 tới N . Cần trả lời M truy vấn. Mỗi truy vấn cho ba số nguyên L_i, R_i, K_i , hỏi trong đoạn $[L_i, R_i]$ có bao nhiêu số xuất hiện đúng K_i lần.

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$.

Phân tích bài toán. Chia dãy thành khối, đặt kích thước mỗi khối là $O(\sqrt{N})$. Số khối cũng là $O(\sqrt{N})$.

Đọc toàn bộ truy vấn một lần và phân loại truy vấn theo khối chứa đầu trái L_i , sau đó xử lý riêng từng loại.

Xét mọi truy vấn có đầu trái nằm trong khối A_i . Sắp xếp các truy vấn này theo vị trí đầu phải trong dãy từ nhỏ tới lớn. Sau đó quét lần lượt mọi phần tử từ đầu trái của khối A_i tới cuối dãy, dùng mảng duy trì tại từng thời điểm số lượng các giá trị có tần suất 1 tới N trong các phần tử đã quét, và tần suất của từng giá trị từ 1 tới N .

Với mỗi truy vấn $[L_i, R_i]$, khi quét tới đầu phải R_i , ta có thể trong $O(\sqrt{N})$ thời gian chuyển thông tin đang duy trì thành thông tin của đoạn $[L_i, R_i]$. Sau khi ghi đáp án của truy vấn, khôi phục thông tin. Mỗi truy vấn được xử lý đúng một lần.

Định lý 4.1. Thuật toán này có độ phức tạp thời gian $O((N + M)\sqrt{N})$ và độ phức tạp không gian $O(N)$.

Trong ví dụ này, ta dùng cách ngoại tuyến để giải quyết vấn đề dung lượng ghi thông tin quá lớn của ví dụ ba. Có thể thấy kết hợp chia khối với thuật toán ngoại tuyến có phần tương tự phương pháp tiền xử lý; hơn nữa, những bài có thể giải bằng chia khối và tiền xử lý thường có thể giảm độ phức tạp không gian trong tình huống ngoại tuyến.

Ví dụ sáu. Cho một cây N đỉnh. Mỗi đỉnh có trọng số là một số nguyên không âm nhỏ hơn 2^{31} . Cần trả lời M truy vấn; mỗi truy vấn hỏi số lượng trọng số đỉnh phân biệt xuất hiện trên đường đi đơn từ đỉnh u_i tới đỉnh v_i .

Quy mô dữ liệu: $1 \leq N \leq 40000, 1 \leq M \leq 100000$. Bài gốc là *Count On A Tree II* trên Sphere Online Judge.

Phân tích bài toán. Trước hết rời rạc hóa mọi trọng số của đỉnh; sau đó có thể xem trọng số là số nguyên không vượt quá N . Dùng phương pháp đánh dấu đỉnh then chốt trên cây đã giới thiệu ở mục 3.3. Đặt $S = \Theta(\sqrt{N})$; khi đó số đỉnh then chốt cũng là $O(\sqrt{N})$. Theo hệ quả 3.11, khoảng cách từ mỗi đỉnh tới đỉnh then chốt gần nhất là $O(\sqrt{N})$.

Đọc toàn bộ truy vấn và phân loại mỗi truy vấn theo đỉnh then chốt gần nhất với đầu nút u_i tương ứng. Sau đó xử lý riêng từng loại.

Xét một loại truy vấn có chung đỉnh then chốt r_i , là đỉnh then chốt gần nhất với các đầu mút u_i . Lấy r_i làm gốc và DFS toàn bộ cây; đồng thời duy trì, trong các đỉnh đã vào stack nhưng chưa ra stack, số lần xuất hiện của từng trọng số và số lượng trọng số phân biệt.

Bổ đề 4.2. Khi đang thăm một đỉnh x_i , mọi đỉnh đã vào stack nhưng chưa ra stack chính là toàn bộ đỉnh trên đường từ x_i tới gốc.

Bổ đề 4.3. Đường đi từ một đỉnh u tới một đỉnh v tương đương với đường từ u tới gốc cộng với đường từ v tới gốc, rồi trừ đường từ $LCA(u, v)$ tới gốc.

Với mỗi truy vấn (u_i, v_i) , khi DFS tới v_i , thông tin trong stack chính là thông tin của các đỉnh trên đường từ v_i tới gốc. Lúc này liệt kê từng đỉnh x trên đường từ u_i tới gốc r_i . Nếu x là $LCA(u_i, v_i)$ thì không thay đổi; nếu x nằm trên đường từ $LCA(u_i, v_i)$ tới gốc r_i thì trừ trọng số của x ; ngược lại cộng trọng số của x . Sau khi thu được đáp án, hoàn tác các thay đổi tạm thời.

Bổ đề 4.4. Có thể trả lời một truy vấn trong $O(\sqrt{N})$ thời gian.

Định lý 4.5. Bài toán này có thể giải trong $O((N + M)\sqrt{N})$ thời gian và $O(N)$ không gian.

8.5 Tổng kết

Nói một cách hình tượng, bản chất của tư tưởng chia khối là chuyển một thuật toán đơn giản có độ phức tạp $O(f(N))$ thành một thuật toán có độ phức tạp

$$O\left(g(S) + \frac{N}{S}h(S)\right),$$

trong đó S là kích thước khối, còn N/S là số khối. Với các bài khác nhau, biểu hiện cụ thể của điều này cũng khác nhau.

Lấy ví dụ một làm mẫu: thuật toán tham khảo của bài đó thể hiện tư tưởng chia khối ở hai chỗ. Một chỗ là dùng hai cấu trúc dữ liệu khác nhau để duy trì các sửa có $D_i < \sqrt{N}$ và các sửa có $D_i \geq \sqrt{N}$. Chỗ còn lại là khi thao tác sửa rất thường xuyên, ta dùng cấu trúc chia khối để duy trì tổng đoạn của dãy. Trong bài đặc biệt này, cách đó hiệu quả hơn cây đoạn, vì số lượng thao tác sửa có thể đạt tới $O(\sqrt{N})$ lần số lượng truy vấn. Các ví dụ sau cũng tương tự. Tuy nhiên, có khi ta chia một bài toán thành nhiều bài con có bản chất giống nhau, có khi lại tách một bài toán thành nhiều bài con khác nhau khá lớn.

Bài viết đưa ra các mô hình cơ bản của chia khối trên dãy tuyến tính và cấu trúc cây; chúng có hiệu lực với một lớp bài rất rộng. Nhưng áp dụng tư tưởng chia khối như thế nào còn tùy từng bài cụ thể. Các bạn không nên bị bó buộc bởi vài hình thức được nêu ở đây, mà cần mạnh dạn sáng tạo. Tất nhiên, nếu tồn tại thuật toán hiệu quả hơn chia khối, cũng phải biết lựa chọn, không nên bảo thủ. Hy vọng các ví dụ này đem lại gợi ý cho người đọc.

8.6 Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 2nd edition.
2. Jon Kleinberg, Eva Tardos. *Algorithm Design*.
3. Wu Wenhui, Wang Jiande. *Giáo trình nâng cao cuộc thi lập trình sinh viên quốc tế ACM/ICPC*, tập 1.

4. Zheng Dun. *Quy hoạch cân bằng: phân tích ứng dụng của một lớp tư tưởng cân bằng*.
5. Qi Zichao. *Ứng dụng thuật toán chia để trị trong các bài toán đường đi trên cây*.
6. Tài liệu về *Candy Park*, NOI Winter Camp 2013.
7. Báo cáo ra đề *JZPLCM*.
8. Xu Jie. Báo cáo ra đề *Đồ thị trong đồ thị*.
9. Xu Haoran. *Bàn rộng về cấu trúc dữ liệu*.

8.7 Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy hướng dẫn Gong Zhiyong đã quan tâm và giúp đỡ tác giả trong nhiều năm.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Hu Weidong và Tang Wenbin vì sự giúp đỡ.

Đặc biệt cảm ơn bạn Fan Haoqiang ở Bắc Kinh đã kiên nhẫn giúp đỡ tác giả trong quá trình viết bài. Tác giả học được rất nhiều và nhận được không ít gợi ý qua các trao đổi với bạn ấy.

Xin cảm ơn đàn anh Huang Jiyuan vì sự giúp đỡ.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ hoặc gợi ý cho tác giả.

Cuối cùng, xin cảm ơn cha mẹ tôi vì sự quan tâm và chăm sóc chu đáo.

9 Kỹ thuật *meet in the middle* trong bài toán tìm kiếm

Qiao Mingda

Trường Ngoại ngữ Nam Kinh, Giang Tô

9.1 Tóm tắt

Meet in the middle là một kỹ thuật tối ưu hóa tìm kiếm thường gặp. Một số bài toán tìm kiếm có thể dùng kỹ thuật này để giảm mạnh độ phức tạp thời gian, nhờ đó tìm được kết quả trong giới hạn thời gian.

Bài viết gồm ba phần. Phần thứ nhất đưa ra một mô hình đồ thị có hướng trừu tượng, dựa vào đó giới thiệu tư tưởng của kỹ thuật *meet in the middle*, và dùng kỹ thuật này để thiết kế thuật toán cho mô hình đồ thị có hướng. Phần thứ hai chứng minh ngắn gọn tính đúng đắn, độ phức tạp thời gian và điều kiện sử dụng của thuật toán trong phần thứ nhất. Phần thứ ba khảo sát vài bài ví dụ dùng kỹ thuật này; từ các ví dụ đó khái quát thành một mô hình phương trình, rồi phân tích một số biến thể của kỹ thuật.

9.2 Phần thứ nhất: mô hình đồ thị có hướng

9.2.1 Bản chất bài toán

Bản chất của một phần các bài toán tìm kiếm là: trong đồ thị có hướng G , tính số đường đi khác nhau có độ dài L từ đỉnh A tới đỉnh B . Ở đây, một “đường đi độ dài L ” được định nghĩa là một dãy cạnh có hướng

$$(E_1, E_2, \dots, E_L),$$

trong đó điểm đầu của E_1 là A , điểm cuối của E_L là B , và với mọi $i = 1, 2, \dots, L - 1$, điểm cuối của E_i bằng điểm đầu của E_{i+1} . Hai đường đi

$$(E_1, E_2, \dots, E_L) \quad \text{và} \quad (E'_1, E'_2, \dots, E'_L)$$

khác nhau khi và chỉ khi tồn tại $1 \leq i \leq L$ sao cho $E_i \neq E'_i$.

Giả sử bậc ra lớn nhất của một đỉnh trong G là hằng số D , không gian cần để mô tả một đỉnh là M , và thời gian cần để tính một hậu kế xác định của một đỉnh là T .

Ví dụ, mỗi đỉnh có nhãn là một dãy độ dài N ,

$$P_1, P_2, \dots, P_N.$$

Từ một đỉnh, ta có thể lật dãy $P_1, P_2, \dots, P_{N-1}$ hoặc dãy $P_2, P_3, \dots, P_N$ để thu được hậu kế. Ở đây, “lật $P_1, P_2, \dots, P_{N-1}$ ” nghĩa là đổi chỗ P_1 với P_{N-1} , P_2 với P_{N-2} , v.v., cho tới $P_{\lfloor N/2 \rfloor}$ và $P_{\lfloor N/2 \rfloor + 1}$. Như vậy mỗi đỉnh có đúng hai hậu kế; mô tả một đỉnh cần $O(N)$ không gian để lưu dãy; tính một hậu kế cần $O(N)$ thời gian để lật dãy. Trong ví dụ này $D = 2$, $M = O(N)$, và $T = O(N)$.

Theo các giả thiết trên, ta phân tích thuật toán DFS thô để giải bài toán này. Thuật toán giới hạn độ sâu DFS không vượt quá L ; khi độ sâu tìm kiếm đạt L , nó so sánh đỉnh hiện tại với B , nếu là cùng một đỉnh thì tăng đáp án thêm 1. Để chứng minh số đỉnh đi qua trong quá trình tìm kiếm không vượt quá

$$1 + D + D^2 + \dots + D^L = \frac{D^{L+1} - 1}{D - 1} = O(D^L),$$

và số đỉnh đạt được sau khi tìm L bước không vượt quá $D^L = O(D^L)$. Lưu ý rằng nếu cùng một đỉnh được đi qua nhiều lần thì phải tính nhiều lần.

Vì một đỉnh chiếm $O(M)$ không gian, so sánh một đỉnh có phải B hay không cần $O(M)$ thời gian. Mặt khác, trong quá trình tìm kiếm, tính mỗi hậu kế cần $O(T)$ thời gian. Vì vậy độ phức tạp thời gian của DFS thô là

$$O(D^L(M + T)).$$

Khi L lớn, thời gian chạy của thuật toán thô sẽ rất dài; trong đa số tình huống nó không thể tính ra đáp án trong giới hạn thời gian. Ta cần xét cách tối ưu hóa tìm kiếm.

Trong lớp bài này, số đỉnh của G rất lớn, thậm chí G có thể là đồ thị vô hạn, nên số bài toán con trùng lặp trong quá trình tìm kiếm khá ít; quy hoạch động không phải một phương pháp hiệu quả. Vì trong trường hợp xấu nhất ta buộc phải duyệt cả cây tìm kiếm, các kỹ thuật tối ưu hóa tìm kiếm nghiệm tối ưu cũng không phù hợp với loại bài đếm số đường đi này. Ta cần thay đổi cách tìm kiếm ở mức tổng thể.

9.2.2 Tư tưởng *meet in the middle*

Nghĩa đen của *meet in the middle* là “gặp nhau ở giữa”. Giả sử có hai người, một người xuất phát từ A , người kia xuất phát từ B , cùng đi về phía điểm xuất phát của người còn lại. Nếu khi cả hai đều đi được $L/2$ mét họ vừa đúng “gặp nhau ở giữa”, thì có thể khẳng định ta đã tìm được một đường đi từ A tới B có độ dài L mét.

Dưới đây ta xét cách áp dụng tư tưởng này vào mô hình đồ thị có hướng.

9.2.3 Quy trình thuật toán cho mô hình đồ thị có hướng

Để thuận tiện mô tả, ta dựng một đồ thị mới G' . Các đỉnh của G' giống với G , nhưng mỗi cạnh có hướng từ X tới Y trong G tương ứng với một cạnh có hướng từ Y tới X trong G' . Ngoài ra, đặt

$$L_1 = \left\lfloor \frac{L}{2} \right\rfloor, \quad L_2 = \left\lceil \frac{L}{2} \right\rceil.$$

Xét riêng tính chẵn lẻ của L sẽ dễ chứng minh $L_1 + L_2 = L$.

Phần thứ nhất của thuật toán thực hiện DFS trong G từ đỉnh A , giới hạn độ sâu tìm kiếm không vượt quá L_1 . Ta ghi lại toàn bộ các đỉnh đạt được sau đúng L_1 bước và số lần đạt được từng đỉnh. Ký hiệu số lần đi L_1 bước tới đỉnh P là $\text{Count}(P)$. Nếu sau L_1 bước chưa từng tới P , quy ước $\text{Count}(P) = 0$.

Phần thứ hai của thuật toán thực hiện DFS trong G' từ đỉnh B , giới hạn độ sâu tìm kiếm không vượt quá L_2 . Nếu sau đúng L_2 bước tìm kiếm tới đỉnh Q , ta tuyên bố đã tìm được $\text{Count}(Q)$ đường đi có độ dài $L_1 + L_2 = L$, và cộng $\text{Count}(Q)$ vào đáp án. Lưu ý rằng sau L_2 bước tìm kiếm ta có thể tới cùng một đỉnh nhiều lần; mỗi lần tới đều phải cộng $\text{Count}(Q)$.

Từ quy trình trên có thể thấy, *meet in the middle* là một kỹ thuật “tìm kiếm hai chiều”.

9.3 Phần thứ hai: phân tích thuật toán

9.3.1 Tính đúng đắn

Đặt

$$L_1 = \left\lfloor \frac{L}{2} \right\rfloor, \quad L_2 = \left\lceil \frac{L}{2} \right\rceil.$$

Với một đỉnh P trong G , gọi $f(P)$ là số đường đi từ A tới P có độ dài L_1 , và gọi $g(P)$ là số đường đi từ P tới B có độ dài L_2 .

Bổ đề 1. Số đường đi từ A tới B có độ dài L là

$$\sum_P f(P)g(P).$$

Chứng minh. Xét một đường đi bất kỳ có độ dài L ,

$$(E_1, \dots, E_L),$$

trong đó E_1 bắt đầu ở A , còn E_L kết thúc ở B . Lấy P là điểm cuối của E_{L_1} ; ta nói đường đi này “đi qua” P .

Ngược lại, nếu chọn một đường đi từ A tới P độ dài L_1 , rồi chọn một đường đi từ P tới B độ dài L_2 , ghép hai dãy cạnh theo thứ tự thì thu được một đường đi từ A tới B độ dài L . Vì vậy, với một đỉnh P cố định, số đường đi “đi qua” P là $f(P)g(P)$. Lấy tổng trên mọi P , ta được số đường đi cần tính.

Bổ đề 2. Thuật toán trong phần thứ nhất cho ra kết quả

$$\sum_P f(P)g(P).$$

Chứng minh. Theo hai lần DFS, với mọi đỉnh P , trong phần thứ nhất của thuật toán ta có $\text{Count}(P) = f(P)$; trong phần thứ hai, số lần tìm kiếm sau L_2 bước đạt tới P là $g(P)$.

Do đó, với các đỉnh có $g(P) = 0$, thuật toán không tăng đáp án, tương đương cộng thêm $f(P)g(P) = 0$. Với các đỉnh có $g(P) > 0$, thuật toán cộng $f(P)$ vào đáp án đúng $g(P)$ lần, tức cộng tổng cộng $f(P)g(P)$. Cộng theo mọi đỉnh P , kết quả của thuật toán là $\sum_P f(P)g(P)$.

Định lý 1. Thuật toán trong phần thứ nhất có thể tính đúng tổng số đường đi khác nhau từ đỉnh A tới đỉnh B có độ dài L trong đồ thị G .

Chứng minh. Kết hợp Bổ đề 1 và Bổ đề 2, đáp án thuật toán đưa ra và số đường đi thực tế đều bằng $\sum_P f(P)g(P)$.

9.3.2 Độ phức tạp thời gian

Dưới đây ta phân tích độ phức tạp thời gian của thuật toán. Ở trên ta đã giả sử bậc ra lớn nhất của một đỉnh trong G là D ; bây giờ giả sử thêm rằng bậc ra của các đỉnh trong G' cũng không vượt quá D . Mặt khác, giống mô tả trước đó, ta vẫn giả sử trong cả G và G' , không gian mô tả một đỉnh là M , và thời gian tính hậu kế xác định của một đỉnh là T .

Độ phức tạp thời gian của thuật toán chủ yếu phụ thuộc vào hai phần: thời gian DFS và thời gian chương trình tính, truy vấn $\text{Count}(P)$. Trong phân tích dưới đây, vì D là hằng số, ta có thể bỏ qua các dấu làm tròn trên số mũ, tức xem

$$O(D^{\lceil L/2 \rceil}) = O(D^{\lfloor L/2 \rfloor}) = O(D^{L/2}).$$

Trong phần thứ nhất, chi phí DFS là $O(D^{L/2}T)$; trong phần thứ hai, chi phí DFS cũng là $O(D^{L/2}T)$. Vì vậy tổng thời gian dùng cho DFS là

$$O(D^{L/2}T) + O(D^{L/2}T) = O(D^{L/2}T).$$

Trong phần thứ nhất, ta cần thống kê số lần $\text{Count}(P)$ của mỗi đỉnh P đạt được sau L_1 bước; trong phần thứ hai, ta cần truy vấn $\text{Count}(Q)$ với mỗi đỉnh Q đạt được sau L_2 bước. Điều này đòi hỏi một cấu trúc dữ liệu có thể chèn dữ liệu nhanh và truy vấn nhanh số lần xuất hiện của một dữ liệu. Có thể thấy bảng băm là lựa chọn lý tưởng. Theo tính chất của bảng băm, nếu dùng băm đều đơn giản (*simple uniform hashing*) và dùng danh sách liên kết để xử lý va chạm, thì trung bình một thao tác chèn hoặc truy vấn chỉ cần $O(M)$ thời gian. Nguyên nhân là trong bài toán này, so sánh hai đỉnh có giống nhau hay không cần $O(M)$ thời gian.

Trong phần thứ nhất, ta cần thực hiện $O(D^{L/2})$ thao tác chèn vào bảng băm; trong phần thứ hai, ta cần thực hiện $O(D^{L/2})$ thao tác truy vấn. Vì vậy thời gian tính và truy vấn $\text{Count}(P)$ là

$$[O(D^{L/2}) + O(D^{L/2})] \cdot O(M) = O(D^{L/2}M).$$

Kết hợp hai phần, độ phức tạp thời gian của thuật toán là

$$O(D^{L/2}(M + T)).$$

9.3.3 Điều kiện sử dụng thuật toán

Muốn dùng thuật toán trong phần thứ nhất cho một bài toán, trước hết bài toán đó phải phù hợp với mô hình đồ thị có hướng. Tiếp theo, trong quá trình phân tích độ phức tạp thời gian của thuật toán, thực ra ta đã thêm một số điều kiện hạn chế.

Trong phân tích trước đó, ta giả sử bậc ra của các đỉnh trong G' không vượt quá bậc ra lớn nhất trong G . Mặt khác, bậc ra của một đỉnh trong G' thực chất bằng bậc vào của đỉnh đó trong G . Vì vậy thông thường cần bảo đảm bậc vào và bậc ra của các đỉnh trong G cùng bậc độ lớn.

Mặt khác, ta giả sử có thể truy cập một hậu kế xác định của một đỉnh trong G' trong $O(T)$ thời gian. Điều kiện này tương đương với yêu cầu có thể truy cập một tiền nhiệm xác định của một đỉnh trong G trong $O(T)$ thời gian.

Về chi phí không gian, để bảo đảm bảng băm có độ phức tạp thao tác kỳ vọng $O(M)$, ta cần tiêu tốn

$$O(D^{L/2}M)$$

không gian.

9.4 Phần thứ ba: ứng dụng

9.4.1 Ví dụ 1: số nghiệm của phương trình

Nội dung bài toán. Cho các số nguyên $M, k_1, k_2, \dots, k_N$ và $p_1, p_2, \dots, p_N$. Hãy tính số nghiệm nguyên của phương trình

$$\sum_{i=1}^N k_i x_i^{p_i} = 0$$

thỏa mãn $1 \leq x_i \leq M$. Bài gốc là NOI 2001, vòng hai.

Quy mô dữ liệu.

$$N \leq 6, \quad M \leq 150.$$

Phân tích. Ta có thể chuyển bài này thành mô hình đồ thị có hướng, rồi dùng thuật toán trong phần thứ nhất để giải.

Dùng cặp (Depth, Sum) để biểu diễn một đỉnh trong G , nên mô tả một đỉnh chỉ cần $O(1)$ không gian. Ở đây Depth biểu diễn số hạng đã xử lý, còn Sum biểu diễn tổng của Depth số hạng đầu. Đỉnh xuất phát là $(0, 0)$, đỉnh kết thúc là $(N, 0)$.

Với một đỉnh (Depth, Sum) có $\text{Depth} < N$, đặt

$$k' = k_{\text{Depth}+1}, \quad p' = p_{\text{Depth}+1}.$$

Từ đỉnh này có M hậu kế:

$$\begin{aligned} &(\text{Depth}+1, \text{Sum} + k' \cdot 1^{p'}), \\ &(\text{Depth}+1, \text{Sum} + k' \cdot 2^{p'}), \\ &\dots, \\ &(\text{Depth}+1, \text{Sum} + k' \cdot M^{p'}). \end{aligned}$$

Nếu xét đồ thị đảo, thì với một đỉnh (Depth, Sum) có $\text{Depth} > 0$, nó có M tiền nhiệm. Đặt

$$k'' = k_{\text{Depth}}, \quad p'' = p_{\text{Depth}}.$$

Các tiền nhiệm là

$$\begin{aligned} &(\text{Depth}-1, \text{Sum} - k'' \cdot 1^{p''}), \\ &(\text{Depth}-1, \text{Sum} - k'' \cdot 2^{p''}), \\ &\dots, \\ &(\text{Depth}-1, \text{Sum} - k'' \cdot M^{p''}). \end{aligned}$$

Để tính một hậu kế hoặc tiền nhiệm của một đỉnh, ta cần tính một lũy thừa. Dùng lũy thừa nhanh thì độ phức tạp thời gian là $O(\log \text{MaxP})$, trong đó MaxP là giá trị lớn nhất trong $p_1, p_2, \dots, p_N$.

Qua phân tích trên, ta có thể dùng trực tiếp thuật toán trong phần thứ nhất, với độ phức tạp thời gian

$$O(M^{N/2} \log \text{MaxP}).$$

Hãy nhìn lại ý nghĩa thực tế của thuật toán trên trong ví dụ này. Xét trường hợp $N = 6$:

$$k_1 x_1^{p_1} + k_2 x_2^{p_2} + k_3 x_3^{p_3} + k_4 x_4^{p_4} + k_5 x_5^{p_5} + k_6 x_6^{p_6} = 0.$$

Thuật toán thô là liệt kê sáu ẩn ở vế trái rồi kiểm tra kết quả của vế trái có bằng hằng số ở vế phải hay không; độ phức tạp thời gian như vậy là $O(M^6 \log \text{MaxP})$, rất kém.

Suy nghĩ kỹ hơn, phương trình trên có vẻ rất mất cân bằng: cả sáu ẩn đều nằm ở vế trái, khiến ta buộc phải liệt kê sáu ẩn. Chuyển về phương trình, ta có

$$k_1x_1^{p_1} + k_2x_2^{p_2} + k_3x_3^{p_3} = -k_4x_4^{p_4} - k_5x_5^{p_5} - k_6x_6^{p_6}.$$

Lúc này hai vế đã “cân bằng”. Giả sử ta liệt kê trước ba số ở vế trái và lưu mọi kết quả liệt kê được vào một bảng, rồi liệt kê ba số ở vế phải. Mỗi khi tính được một kết quả trùng với một số trong bảng, ta thu được một nghiệm của phương trình. Khi so sánh kết quả tính toán, ta không cần so sánh từng cặp số; có thể dùng cấu trúc dữ liệu, chẳng hạn bảng băm, để tăng tốc việc thống kê. Từ đó ta có thuật toán độ phức tạp $O(M^3 \log \text{MaxP})$. Thực chất, điểm then chốt của *meet in the middle* nằm ở việc giảm lượng tính toán bằng cách tối ưu hóa quá trình hợp nhất.

9.4.2 Ví dụ 2: *ABCDEF*

Nội dung bài toán. Cho một tập số nguyên S . Hãy tính có bao nhiêu bộ (a, b, c, d, e, f) thỏa mãn $a, b, c, d, e, f \in S, d \neq 0$, và

$$ab + c = d(e + f).$$

Bài gốc là *ABCDEF* trên SPOJ.

Quy mô dữ liệu.

$$|S| \leq 100.$$

Phân tích. Bài này chỉ cần chuyển về phương trình thành

$$ab + c = d(e + f),$$

vì vậy về bản chất nó giống ví dụ 1. Ta thu được thuật toán có độ phức tạp

$$O(|S|^3).$$

9.4.3 Ví dụ 3: *EllysBulls*

Nội dung bài toán. Có một số thập phân N chữ số A , có thể có chữ số 0 ở đầu. Bây giờ cho M điều kiện; điều kiện thứ i gồm một số N chữ số X_i và giá trị $\text{Same}(A, X_i)$. Ở đây $\text{Same}(A, B)$ biểu diễn số vị trí mà hai số A và B có chữ số giống nhau. Ví dụ,

$$\text{Same}(\text{"0312"}, \text{"0321"}) = 2.$$

Nếu có duy nhất một A thỏa mãn yêu cầu thì in nghiệm đó; ngược lại in thông tin biểu thị vô nghiệm hoặc nhiều nghiệm. Bài gốc là TopCoder Single Round Match 572, Division I, Level Two.

Quy mô dữ liệu.

$$N \leq 9, \quad M \leq 50.$$

Phân tích. Để tiện trình bày, định nghĩa vector hàng

$$\text{Sum} = (\text{Same}(A, X_1), \text{Same}(A, X_2), \dots, \text{Same}(A, X_M)).$$

Gọi $X_{i,j}$ là chữ số thứ j của X_i . Định nghĩa hàm $S(a, b)$ với $0 \leq a, b \leq 9$: nếu $a = b$ thì $S(a, b) = 1$, nếu $a \neq b$ thì $S(a, b) = 0$. Tiếp tục định nghĩa

$$\text{Cost}(i, j) = (S(X_{1,i}, j), S(X_{2,i}, j), \dots, S(X_{M,i}, j)).$$

Vector này cho biết: nếu chữ số thứ i của A là j , thì riêng vị trí này khớp với những điều kiện nào.

Theo điều kiện của đề bài, ta có phương trình

$$\text{Cost}(1, a_1) + \text{Cost}(2, a_2) + \dots + \text{Cost}(N, a_N) = \text{Sum}.$$

Ta vẫn chuyển về giống hai bài trước. Đặt

$$\text{Mid} = \left\lfloor \frac{N}{2} \right\rfloor,$$

thu được

$$\begin{aligned} & \text{Cost}(1, a_1) + \dots + \text{Cost}(\text{Mid}, a_{\text{Mid}}) \\ &= \text{Sum} - \text{Cost}(\text{Mid} + 1, a_{\text{Mid} + 1}) - \dots - \text{Cost}(N, a_N). \end{aligned}$$

Vì mỗi vị trí có 10 khả năng, nên tìm kiếm về trái có $O(10^{N/2})$ kết quả khác nhau, và về phải cũng có $O(10^{N/2})$ kết quả khác nhau. Hai vế của phương trình đều là vector hàng $1 \times M$, nên mỗi bước tính toán và thao tác trong bảng băm cần $O(M)$ thời gian. Vì vậy ta có thuật toán độ phức tạp $O(10^{N/2}M)$, đủ để vượt qua dữ liệu kiểm thử của bài này.

9.4.4 Tiểu kết: mô hình phương trình

Tổng kết ba ví dụ trên, ta thấy các bài toán đều có thể chuyển thành một mô hình phương trình:

$$\sum_{i=1}^N f(i, x_i) = S.$$

Ở ví dụ 1 và 2, $f(i, x_i)$ và S là số nguyên; ở ví dụ 3, $f(i, x_i)$ và S là vector. Trong cả ba bài, cách làm đều dựa trên việc biến đổi phương trình thành

$$\sum_{i=1}^{\lfloor N/2 \rfloor} f(i, x_i) = S - \sum_{i=\lfloor N/2 \rfloor + 1}^N f(i, x_i).$$

Ta tìm kiếm về trái trước và ghi kết quả vào bảng băm; sau đó tìm kiếm về phải và truy vấn các kết quả đã lưu trong bảng băm. Có thể thấy quy trình giải ngắn gọn này rất giống thuật toán trong phần thứ nhất. Hơn nữa, từ phân tích của ví dụ 1, mô hình phương trình có thể chuyển thành mô hình đồ thị có hướng trong phần thứ nhất, nên mô hình phương trình là một trường hợp đặc biệt của mô hình đồ thị có hướng. Với các bài thuộc mô hình phương trình, ta dễ nghĩ tới kỹ thuật *meet in the middle*.

Vận dụng hai mô hình trên, ta có thể nhanh chóng mô hình hóa bài toán cụ thể và dùng kỹ thuật *meet in the middle* để giải. Tuy nhiên, một số bài không thể chuyển thành mô hình phương trình hoặc mô hình đồ thị có hướng, do đó không thể dùng trực tiếp thuật toán của phần thứ nhất. Dù vậy, chúng vẫn có điểm tương tự với hai mô hình trên; ta vẫn có thể sửa thuật toán trong phần thứ nhất để dùng kỹ thuật *meet in the middle* tối ưu các bài này. Dưới đây ta phân tích vài ví dụ như vậy.

9.4.5 Ví dụ 4: *Balanced Cow Subsets*

Nội dung bài toán. Cho các số nguyên dương $a_1, \dots, a_N$. Trước hết chọn ra một số số trong N số, ít nhất một số; sau đó chia các số đã chọn thành hai phần sao cho tổng của hai phần bằng nhau. Hãy tính số phương án chọn số ở bước đầu.

Lưu ý: có thể tồn tại một phương án chọn số cho phép nhiều cách chia khác nhau, nhưng phương án chọn số đó chỉ được tính một lần. Bài gốc là USACO 2012 US Open, Gold Division.

Quy mô dữ liệu.

$$N \leq 20.$$

Phân tích. Ta đổi bài toán sang một cách mô tả tương đương: với dãy $a_1, a_2, \dots, a_N$, đặt $x_i \in \{-1, 0, 1\}$. Dưới điều kiện

$$\sum_{i=1}^N x_i a_i = 0,$$

cần tính có bao nhiêu tập khác nhau, không rỗng,

$$S = \{i \mid 1 \leq i \leq N, x_i \neq 0\}.$$

Bài này giống mô hình phương trình đã nêu, nhưng điều ta cần không phải số nghiệm của phương trình, mà là số tập tương ứng với các nghiệm của phương trình. Vì vậy thuật toán cần sửa đôi chút.

Định nghĩa một dãy mới $y_1, y_2, \dots, y_N$: nếu $x_i = 0$ thì $y_i = 0$, còn nếu $x_i \neq 0$ thì $y_i = 2^{i-1}$. Khi đó tập S có thể biểu diễn bằng $\sum y_i$.

Chọn một số nguyên M trong đoạn $[0, N]$, biến đổi $\sum x_i a_i = 0$ thành

$$x_1 a_1 + x_2 a_2 + \dots + x_M a_M = -x_{M+1} a_{M+1} - x_{M+2} a_{M+2} - \dots - x_N a_N.$$

Ký hiệu vế trái là LeftSum, vế phải là RightSum. Tương tự, ký hiệu

$$\text{LeftSet} = y_1 + y_2 + \dots + y_M, \quad \text{RightSet} = y_{M+1} + y_{M+2} + \dots + y_N.$$

Trong phần thứ nhất của thuật toán, ta vẫn tìm kiếm M ở vế trái như các bài trước, tổng cộng có 3^M trường hợp. Ta không chỉ ghi LeftSum, mà còn ghi cả LeftSet tương ứng.

Trong phần thứ hai của thuật toán, nếu phát hiện RightSum bằng một LeftSum nào đó, thì tập được biểu diễn bởi

$$\text{LeftSet} + \text{RightSet}$$

là một tập hợp hợp lệ. Ta có thể dùng một mảng boolean đánh số từ 0 tới $2^N - 1$ để ghi nhận các tập này.

Cuối cùng, quét toàn bộ mảng boolean một lần là thống kê được đáp án.

Dễ thấy phần thứ nhất có độ phức tạp thời gian $O(3^M)$. Phần thứ hai liệt kê $O(3^{N-M})$ trường hợp, nhưng với mỗi trường hợp cần truy cập tất cả LeftSet tương ứng. Trong trường hợp xấu nhất, ta cần truy cập $O(2^M)$ LeftSet, nên độ phức tạp thời gian của phần thứ hai là $O(3^{N-M} 2^M)$. Thời gian quét cuối cùng là $O(2^N)$.

Tóm lại, độ phức tạp thời gian là

$$O(3^M + 3^{N-M} 2^M + 2^N) = O(3^M + 3^{N-M} 2^M).$$

Nếu chọn $M = \lfloor N/2 \rfloor$, ta thu được

$$O(6^{N/2}) \approx O(2.45^N),$$

đủ để vượt qua dữ liệu kiểm thử của bài này. Nhưng thực tế, nếu chọn

$$M = N \log_{9/2} 3,$$

ta có được độ phức tạp tốt hơn:

$$O((3^{\log_{9/2} 3})^N) \approx O(2.23^N).$$

Ví dụ 4 cho thấy khi dùng kỹ thuật *meet in the middle*, ta có thể lưu thêm một số thông tin khác trong bảng băm để thuận tiện cho việc giải bài. Mặt khác, khi dùng kỹ thuật này, nếu chia số ẩn cần tìm kiểm thành hai phần không đều nhau, đôi khi ta có thể thu được độ phức tạp thời gian tốt hơn.

9.4.6 Ví dụ 5: *AlphabetPaths*

Nội dung bài toán. Cho một ma trận $R \times C$. Mỗi ô của ma trận hoặc là ô trống, hoặc chứa một số nguyên từ 0 tới 20. Hãy tính có bao nhiêu đường đi gồm 21 ô thỏa mãn: hai ô kề nhau trên đường đi phải có một cạnh chung, và mỗi số từ 0 tới 20 xuất hiện đúng một lần trên đường đi. Bài gốc là TopCoder Single Round Match 523, Division I, Level Three.

Quy mô dữ liệu.

$$R, C \leq 21.$$

Phân tích. Bài này nhìn qua không phải mô hình phương trình đã nêu, mà giống mô hình đồ thị có hướng trong phần thứ nhất. Tuy nhiên, trong bài này điểm đầu và điểm cuối của đường đi đều không xác định, mỗi loại có $O(RC)$ cách chọn. Nếu liệt kê cả điểm đầu và điểm cuối thì ta đã mất $O(R^2C^2)$ lần liệt kê; nhân thêm thời gian tìm kiếm thì hoàn toàn không thể vượt qua.

Ý tưởng vừa nêu là liệt kê điểm đầu và điểm cuối, rồi tìm từ hai đầu đường đi vào giữa. Ta nhận thấy lựa chọn “điểm giữa” thật ra cũng chỉ có $O(RC)$ cách. Vậy tại sao không liệt kê điểm giữa, rồi tìm kiếm từ giữa ra hai đầu?

Do đó ta liệt kê điểm giữa của đường đi, gọi là Mid, tức ô thứ 11. Từ Mid tìm kiếm 10 bước, ta có thể thu được một đường đi P gồm 11 ô. Gọi $\text{Num}(P)$ là tập các giá trị của 10 ô không tính Mid. Gọi giá trị trong Mid là X . Nếu tồn tại hai đường đi P_1 và P_2 thỏa mãn

$$\text{Num}(P_1) \cup \text{Num}(P_2) \cup \{X\} = \{0, 1, \dots, 20\},$$

thì P_1 và P_2 ghép lại là một đường đi thỏa mãn yêu cầu. Chú ý rằng vì các số 0 tới 20 đều xuất hiện đúng một lần, nên có thể khẳng định P_1 và P_2 chỉ giao nhau tại Mid. Giống bài trước, ta dùng một số nhị phân để biểu diễn tập $\text{Num}(P)$, nhờ đó có thể nhanh chóng kiểm tra

$$\text{Num}(P_1) \cup \text{Num}(P_2) \cup \{X\} = \{0, 1, \dots, 20\}.$$

Ước lượng lượng tính toán của thuật toán trên. Trước hết số cách liệt kê Mid là $O(RC)$. Tiếp theo, vì tìm kiếm 10 bước và mỗi bước có thể đi theo bốn hướng, số đường đi là 4^{10} . Thực ra ước lượng này có thể chính xác hơn: từ bước tìm kiếm thứ hai trở đi, “đi ngược lại” hiển nhiên không hợp lệ, nên từ bước thứ hai chỉ còn ba hướng có thể đi. Vì vậy số đường đi thực tế không vượt quá $4 \cdot 3^9$. Với mỗi đường đi, thao tác bảng băm và so sánh đều chỉ cần thời gian hằng số. Tóm lại, ta thu được thuật toán có độ phức tạp

$$O(RC \cdot 4 \cdot 3^9).$$

Dù $RC \cdot 4 \cdot 3^9$ có thể đạt

$$21^2 \cdot 4 \cdot 3^9 = 34720812,$$

nhưng do đường đi không được vượt khỏi biên ma trận hoặc đi tới ô không có số, và nếu một số xuất hiện hai lần trên đường đi thì có thể cắt nhánh ngay, lượng tìm kiếm thực tế sẽ nhỏ hơn rất nhiều so với ước lượng trên. Trong kiểm thử thực tế, chương trình cài đặt theo thuật toán trên vượt qua toàn bộ dữ liệu kiểm thử.

Ví dụ 5 cho thấy khi dùng kỹ thuật *meet in the middle*, nếu điểm đầu và điểm cuối chưa biết, đồng thời số điểm giữa khá ít, ta có thể thử đổi “gặp nhau ở giữa” thành “xuất phát từ giữa”.

9.4.7 Ví dụ 6: *FencingGarden*

Nội dung bài toán. Có N thanh gỗ, độ dài lần lượt là $L_1, L_2, \dots, L_N$. Bạn có thể chọn trước một thanh gỗ và cắt nó thành hai đoạn, độ dài không nhất thiết là số nguyên. Từ các thanh gỗ thu được, chọn một phần để tựa vào một bức tường dài vô hạn và rào một vùng hình chữ nhật. Hãy tìm độ dài cạnh song song với tường của hình chữ nhật khi diện tích vùng hình chữ nhật đạt giá trị lớn nhất. Bài gốc là TopCoder Single Round Match 461, Division I, Level Three.

Quy mô dữ liệu.

$$N \leq 40, \quad L_i \leq 10^8.$$

Phân tích. Bài này khác mọi ví dụ trước: nó không yêu cầu đếm số phương án, mà yêu cầu một nghiệm tối ưu.

Một khó khăn của bài là có rất nhiều cách cắt thanh gỗ; liệt kê cách cắt hiển nhiên là không thực tế.

Vì ta cắt một thanh gỗ thành hai đoạn, còn hình chữ nhật được rào bởi ba cạnh, chắc chắn có một cạnh hoàn toàn được ghép từ các thanh gỗ nguyên vẹn. Cạnh này có hai khả năng: song song với tường hoặc vuông góc với tường. Gọi độ dài cạnh đó là Len .

Trước hết xét trường hợp cạnh này song song với tường. Đặt

$$S = \sum_{i=1}^N L_i.$$

Khi đó hai cạnh còn lại của hình chữ nhật phải có độ dài bằng nhau. Ta xếp các thanh gỗ chưa dùng lại với nhau; tổng độ dài của chúng là $S - Len$. Cắt một nhát tại vị trí $(S - Len)/2$. Nếu vị trí này đúng là chỗ nối của hai thanh gỗ, ta không làm thay đổi số lượng thanh gỗ; nếu đây là một điểm ở giữa một thanh gỗ, ta cắt thanh đó thành hai đoạn. Dù thế nào, ta luôn có thể chia các thanh gỗ còn lại thành hai phần có độ dài đều là $(S - Len)/2$, nhờ đó thu được hình chữ nhật diện tích

$$Len \cdot \frac{S - Len}{2}.$$

Tương tự, nếu một cạnh độ dài Len vuông góc với tường và hoàn toàn được ghép từ các thanh gỗ nguyên vẹn, ta có thể thu được một hình chữ nhật diện tích

$$Len(S - 2Len).$$

Theo tính chất của hàm bậc hai, trong trường hợp thứ nhất $Len = S/2$ là tối ưu; trong trường hợp thứ hai $Len = S/4$ là tối ưu. Vì vậy ta cần tìm, trong các độ dài có thể ghép từ các thanh gỗ nguyên vẹn, những độ dài gần $S/2$ và $S/4$ nhất.

Giống các bài trước, ta vẫn chia N thanh gỗ thành hai phần, lần lượt có M thanh và $N - M$ thanh. Ta tìm kiếm trước M thanh và ghi lại các độ dài thu được. Sau đó tìm kiếm $N - M$ thanh còn lại; giả sử thu được một phần có độ dài T , thì cần tìm trong các độ dài đã tìm trước đó một x sao cho $x + T$ gần $S/2$

hoặc $S/4$ nhất, tức làm cho x gần $S/2 - T$ hoặc $S/4 - T$ nhất. Một cách xử lý tốt là sắp xếp trước $O(2^M)$ độ dài thu được ở phần thứ nhất; khi truy vấn ở phần thứ hai, dùng hai lần tìm kiếm nhị phân.

Sắp xếp $O(2^M)$ số có độ phức tạp thời gian $O(2^M M)$. Thực hiện $O(2^{N-M})$ lần tìm kiếm nhị phân trong đó có độ phức tạp $O(2^{N-M} M)$. Nếu chọn $M = \lfloor N/2 \rfloor$, độ phức tạp thời gian của thuật toán là

$$O(2^{N/2} N).$$

Ví dụ 6 cho thấy kỹ thuật *meet in the middle* không chỉ giải được các bài toán đếm, mà còn có thể kết hợp với tìm kiếm nhị phân và các phương pháp khác để xử lý một số bài tối ưu đặc biệt.

9.5 Tổng kết

Từ các phần trên có thể thấy, *meet in the middle* là một kỹ thuật tối ưu hóa tìm kiếm rất hiệu quả. Tư tưởng cốt lõi của nó là chia bài toán thành hai phần để tìm kiếm riêng, rồi nhanh chóng hợp nhất kết quả của hai phần tìm kiếm nhằm tối ưu thuật toán. Tuy ta không nhờ đó thu được một thuật toán đa thức, nhưng thường có thể làm số mũ trong độ phức tạp thời gian giảm một nửa; trong một số bài, điều này đã đủ để ta tính được kết quả trong giới hạn thời gian.

Ưu điểm của kỹ thuật này là thuật toán dễ cài đặt; quá trình tìm kiếm mà thuật toán dùng về cơ bản giống thuật toán thô, chỉ cần thêm một số cấu trúc dữ liệu khá đơn giản, chẳng hạn bảng băm. Điểm yếu là phạm vi sử dụng không quá rộng, và độ phức tạp thời gian sau tối ưu vẫn là cấp số mũ, nên quy mô dữ liệu có thể xử lý hiệu quả không tăng quá nhiều. Đồng thời, thuật toán phải tiêu tốn rất nhiều không gian.

Mô hình đồ thị có hướng và mô hình phương trình khái quát các đặc trưng phổ biến của những bài mà kỹ thuật *meet in the middle* có thể giải; dùng hai mô hình này có thể giải được một lớp bài toán. Mặt khác, vẫn có những bài không thể chuyển trực tiếp thành các mô hình trên, đòi hỏi ta sửa kỹ thuật *meet in the middle*. Từ đó có thể thấy cùng một kỹ thuật cũng có nhiều cách sử dụng khác nhau trong các bài khác nhau. Ta cần linh hoạt dùng kỹ thuật *meet in the middle* theo tình huống cụ thể để đạt mục đích tối ưu thuật toán.

9.6 Lời cảm ơn

Xin cảm ơn người thân và bạn bè đã động viên, ủng hộ tôi.

Xin cảm ơn các thầy cô Li Shu, Wu Xiaoshi và Shi Zhenlei đã hướng dẫn tôi.

Xin cảm ơn Jia Zhipeng, Xu Haoran và nhiều bạn học thi tin học khác đã giúp đỡ và gợi mở cho tôi.

9.7 Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*.
2. vexorian. TopCoder Algorithm Problem Set Analysis.
3. Bruce Merry. USACO 2012 US Open Gold Division, Balanced Cow Subsets Solution.
4. Zhipeng Jia. TopCoder SRM 450–499 Solution.